```
# Practical 01 --- Your MLOps Workbench

**A clean environment, a pinned library list, and a run that repeat
itself**

SCSE3040 Machine Learning Operations &middot; Bennett University
&middot; Session 2026-27

| | |
|---|---|
| **Lectures this follows** | L01–L02 |
| **Course Outcome** | CO1 |
| **Lab duration** | 120 minutes |
| **Memory needed** | about 250 MB (fine on a 4 GB or 8 GB machine) |
| **Extra software** | nothing beyond the course venv |
| **Marks** | 10 |

---

## Aim

1. Find out which Python is actually running your code.
```

2. Write down the exact library versions your project needs.
3. Make a program that gives the same answer every single time.
4. Save your work in git with a proper first commit.

What you need to know first

MLOps (Machine Learning Operations) is the work of taking a model out of a notebook and keep running for real users. Almost every problem in that job comes from one sentence: *"but it works machine"*.

It works on your machine because your machine has a particular Python, with particular libraries particular versions. Your friend's machine has different ones. The server has different ones agair same code then gives a different answer, or no answer at all.

Professionals solve this in three steps, and today you will do all three.

1. **A virtual environment.** A private folder holding one Python and one set of libraries, used by project only. Installing something for this course then cannot break another project on the sa laptop.
2. **A pinned requirements file.** A plain text list saying *exactly* which version of each library you `numpy==2.5.1`, not just `numpy`. Anybody can then rebuild your environment.
3. **A seed.** Machine learning uses random numbers: which rows go into training, where a mod Random means *different every run*, which means results you cannot check. Fixing the **seed** the randomness repeat, so your result can be checked by someone else.

Together these give you **reproducibility**: same code plus same data plus same versions gives t answer, on any machine, on any day. Everything else in this course is built on top of that.

```
---

## Before you start

- The course virtual environment is installed. If not, follow `labs
SETUP.md` first.
```

- You have opened this notebook from inside the `labs/P01-workbench/` folder.
- Nothing else. This is the first practical of the course.

How to run a cell

Click on a grey code cell, then press **Shift + Enter**. The cell runs and the cursor moves to th one. A number appears in the `[ ]` on the left when the cell has finished.

Run the cells in order, from the top. A later cell almost always uses something an earlier c created. If you jump ahead you will see a `NameError`, which just means "you have not ma thing yet".

If everything goes wrong, use the menu: **Kernel -> Restart Kernel and Clear All Outputs**, start again from the first cell. Nothing is damaged by doing this.

In [1]:
```
# Step 0 --- check the workbench before we start.
# This cell only looks; it changes nothing. Run it and read the last line

import sys
from pathlib import Path

print("Python  :", sys.version.split()[0])
print("Folder  :", Path.cwd().name)

_missing = []
for _name in ['numpy', 'pandas', 'sklearn']:
    try:
        __import__(_name)
    except ImportError:
        _missing.append(_name)

for _name in ['numpy', 'pandas', 'sklearn']:
    _mark = "missing" if _name in _missing else "ok"
    print(f"  {_name:<14} {_mark}")

if _missing:
    print()
    print("STOP. Some libraries are missing:", ", ".join(_missing))
    print("Ask your instructor to run the setup in labs/SETUP.md.")
else:
    print()
    print("All good. You can carry on to Step 1.")
```

```
Python  : 3.14.3
Folder  : P01-workbench
  numpy          ok
  pandas         ok
  sklearn        ok

All good. You can carry on to Step 1.
```

Step 0b --- the dataset

Every practical in this course uses the same 600 food deliveries. The next cell makes sure the fil
there.

In [2]:

```python
# The delivery-time dataset every practical in this course uses.
# If the file is missing we build it again from the same seed, so every
# student in the room gets byte-for-byte the same 600 rows.

import csv
from pathlib import Path

import numpy as np

SEED = 42
N_ROWS = 600
DATA = Path("..") / "data" / "delivery_times.csv"


def make_delivery_csv(path=DATA):
    """Write the 600-row delivery dataset. Same formula as the lectures."
    rng = np.random.default_rng(SEED)
    distance_km = np.round(rng.uniform(0.5, 12.0, N_ROWS), 2)
    prep_time_min = np.round(rng.uniform(5, 30, N_ROWS), 0)
    traffic_level = rng.integers(1, 4, N_ROWS)
    rain = rng.binomial(1, 0.25, N_ROWS)
    delivery_min = np.round(
        6.0
        + 3.1 * distance_km
        + 0.65 * prep_time_min
        + 4.2 * traffic_level
        + 5.5 * rain
        + rng.normal(0, 2.5, N_ROWS),
        1,
    )
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(["distance_km", "prep_time_min", "traffic_level",
                    "rain", "delivery_min"])
        for i in range(N_ROWS):
            w.writerow([distance_km[i], int(prep_time_min[i]),
                        int(traffic_level[i]), int(rain[i]), delivery_min
    return path


if not DATA.exists():
    make_delivery_csv()
```

```
    print("dataset rebuilt ->", DATA)
else:
    print("dataset found   ->", DATA)
```

```
dataset found   -> ../data/delivery_times.csv
```

Walkthrough

Read each step, then run its cell.

Step 1 --- Which Python is running this notebook?

Your laptop probably has more than one Python. There is the one Windows ships, maybe one fr
Anaconda, and the one this course installed. The very first thing to check is which one you are a
using, because that decides which libraries you can see.

In [3]:
```
import sys
from pathlib import Path

print("Python version :", sys.version.split()[0])
print("Python program :", sys.executable)

# A virtual environment is just a folder. If the path above sits
# inside a folder called .venv, you are in the course environment.
in_venv = ".venv" in sys.executable.replace("\\", "/")
print("Inside .venv    :", in_venv)
print("Working folder  :", Path.cwd())
```

```
Python version : 3.14.3
Python program : /Users/archit/Desktop/mlops-lab/SCSE3040-Lab/.venv/bin/pytho
Inside .venv    : True
Working folder  : /Users/archit/Desktop/mlops-lab/SCSE3040-Lab/P01-workbench
```

If `Inside .venv` says `False`, you are running the wrong Python. Tell your instructor before further --- later cells will fail in confusing ways.

Step 2 --- What is installed in this environment?

A library is code somebody else wrote that you can use. Your project depends on several. Each **version number**, and versions matter: `scikit-learn` 1.9 does not behave exactly like 1.2.

Let us ask Python what it has.

In [4]:
```
from importlib.metadata import version

LIBRARIES = ["numpy", "pandas", "scikit-learn", "matplotlib"]

for name in LIBRARIES:
    print(f"{name:<15} {version(name)}")
```

```
numpy           2.5.1
pandas          2.3.3
scikit-learn    1.9.0
matplotlib      3.11.1
```

Write these numbers down somewhere. In six months, when your code suddenly stops working, question anyone will ask is *"which versions were you using?"*

Step 3 --- Freeze those versions into requirements.txt

`requirements.txt` is the standard file name for that list. One library per line. The `==` mear *this version*, which is what "pinned" means.

We will write it into a folder called `work/`, which is where everything you create today will live.

In [5]:
```
WORK = Path("work")
WORK.mkdir(exist_ok=True)

lines = [f"{name}=={version(name)}" for name in LIBRARIES]
(WORK / "requirements.txt").write_text("\n".join(lines) + "\n",
                                       encoding="utf-8")

print("wrote", WORK / "requirements.txt")
print("-" * 40)
print((WORK / "requirements.txt").read_text(encoding="utf-8"))
```

```
wrote work/requirements.txt
----------------------------------------
numpy==2.5.1
pandas==2.3.3
scikit-learn==1.9.0
matplotlib==3.11.1
```

On a new machine, one command rebuilds your whole environment from this file:

```
pip install -r requirements.txt
```

That single line is why "works on my machine" stops being an excuse.

Step 4 --- Random numbers change every time you ask

Now the third idea: randomness. Run the next cell, then run it a second time. Look at the numbe

In [6]:
```
import numpy as np

careless = np.random.default_rng()   # no seed given
print("three random numbers:", np.round(careless.uniform(0, 10, 3), 2))
```

```
three random numbers: [1.42 1.77 7.07]
```

Different every time. That is fine for a dice game and a disaster for a model: your accuracy score change every run and nobody could tell whether your improvement was real.

Step 5 --- A seed makes the randomness repeat

A **seed** is a starting number for the random number generator. Same seed in, same sequence ou
any machine, forever.

`42` is the number everyone uses, for a joke that is older than you.

In [7]:
```
first  = np.random.default_rng(42).uniform(0, 10, 3)
second = np.random.default_rng(42).uniform(0, 10, 3)

print("first run :", np.round(first, 2))
print("second run:", np.round(second, 2))
print("identical :", np.array_equal(first, second))
```

```
first run : [7.74 4.39 8.59]
second run: [7.74 4.39 8.59]
identical : True
```

This is the single most important habit in this course. **Every time you use randomness, set a s**

Step 6 --- Build the delivery dataset, twice

Now we use the seed for something real. The course project predicts how many minutes a food
will take. The dataset has 600 orders and five columns.

We will build it twice and prove the two files are identical, using a **checksum** --- a short fingerpri
file's contents. If one single character differs, the fingerprint changes completely.

In [8]:
```
import hashlib

def sha256_of(path):
    """A short fingerprint of a file's exact contents."""
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()

make_delivery_csv(WORK / "run_a.csv")
make_delivery_csv(WORK / "run_b.csv")

fp_a = sha256_of(WORK / "run_a.csv")
fp_b = sha256_of(WORK / "run_b.csv")

print("run A:", fp_a[:16], "...")
print("run B:", fp_b[:16], "...")
print("identical files:", fp_a == fp_b)
```

```
run A: 9e9f7a46c817d5bb ...
run B: 9e9f7a46c817d5bb ...
identical files: True
```

Two separate runs, byte-for-byte the same file. That is reproducibility you can actually prove to
somebody.

Step 7 --- Look at the data you just made

CSV (Comma Separated Values) is a plain text table. `pandas` is the library that reads one into something you can work with.

In [9]:
```
import pandas as pd

orders = pd.read_csv(DATA)

print("rows, columns:", orders.shape)
print()
print(orders.head())
print()
print(orders.describe().round(1))
```

```
rows, columns: (600, 5)

   distance_km  prep_time_min  traffic_level  rain  delivery_min
0         9.40             17              1     0          51.3
1         5.55             24              2     1          54.2
2        10.37             28              3     0          67.7
3         8.52             23              2     0          51.2
4         1.58             29              2     1          42.1

       distance_km  prep_time_min  traffic_level   rain  delivery_min
count        600.0          600.0          600.0  600.0         600.0
mean           6.2           17.6            2.0    0.3          46.6
std            3.3            7.4            0.8    0.4          12.1
min            0.6            5.0            1.0    0.0          18.5
25%            3.2           11.0            1.0    0.0          37.8
50%            6.2           17.0            2.0    0.0          46.5
75%            9.1           24.0            3.0    1.0          55.7
max           12.0           30.0            3.0    1.0          75.1
```

`distance_km` is how far the rider goes, `prep_time_min` is how long the restaurant takes, `traffic_level` is 1 to 3, `rain` is 0 or 1, and `delivery_min` is the answer we will event to predict.

Step 8 --- Record what you ran

The last habit: write down the facts of the run itself. Months later this file is the only thing that car what produced a result.

In [10]:
```
import json

run_info = {
    "python": sys.version.split()[0],
    "seed": SEED,
    "rows": len(orders),
    "data_sha256": sha256_of(DATA),
    "libraries": {n: version(n) for n in LIBRARIES},
}

(WORK / "run_info.json").write_text(json.dumps(run_info, indent=2),
```

```
                                        encoding="utf-8")
print(json.dumps(run_info, indent=2))
```

```
{
  "python": "3.14.3",
  "seed": 42,
  "rows": 600,
  "data_sha256":
"61bfd2533f407c8ae8a1adcb6e42e1f0fe42cf9c8a6bfe8747673abcafc828c4",
  "libraries": {
    "numpy": "2.5.1",
    "pandas": "2.3.3",
    "scikit-learn": "1.9.0",
    "matplotlib": "3.11.1"
  }
}
```

Step 9 --- Save your work in git

git is the tool that remembers every version of your files. A **commit** is one saved point you can a come back to.

We will make a small repository inside `work/` so nothing else on your machine is touched. Rea output: git prints what it did.

In [11]:
```
import subprocess

def git(*args):
    """Run one git command inside work/ and show what it said."""
    done = subprocess.run(["git", *args], cwd=WORK,
                          capture_output=True, text=True)
    print("$ git", " ".join(args))
    print((done.stdout + done.stderr).strip() or "(no output)")
    print("-" * 50)
    return done

if not (WORK / ".git").exists():
    git("init", "-q")
git("config", "user.name", "SCSE3040 Student")
git("config", "user.email", "student@bennett.edu.in")
git("add", "requirements.txt", "run_info.json")
git("commit", "-q", "-m", "P01: pinned requirements and run record")
git("log", "--oneline")
```

```
$ git init -q
(no output)
--------------------------------------------------
$ git config user.name SCSE3040 Student
(no output)
--------------------------------------------------
$ git config user.email student@bennett.edu.in
(no output)
--------------------------------------------------
$ git add requirements.txt run_info.json
(no output)
--------------------------------------------------
$ git commit -q -m P01: pinned requirements and run record
(no output)
--------------------------------------------------
$ git log --oneline
77326d8 P01: pinned requirements and run record
--------------------------------------------------
```

Out[11]:
```
CompletedProcess(args=['git', 'log', '--oneline'], returncode=0, stdout='773
P01: pinned requirements and run record\n', stderr='')
```

You now have the four habits every later practical assumes: the right environment, pinned versio fixed seed, and a commit. Everything from Docker to Kubernetes is only useful once these are ir

---

Your turn

The walkthrough above is finished. Now you write some code.

There are **3 tasks**. Each one is small. Each one has a hint. Do them in order.

Where you see `# TODO`, replace that line with your own code. Do not delete the variable name left of the `=` sign --- the self-check at the bottom looks for exactly that name.

When you have tried all three, run the **self-check** cell at the end. It prints a table telling you whic are correct. You can run it as many times as you like.

Task T1 --- Change the seed

> The dataset was built with seed **42**. Build it with seed **7** instead, and report the first three `distance_km` values.
>
> Do not change `make_delivery_csv`. Use `np.random.default_rng(7)` directly, exactly the way the function does: draw 600 numbers betwe `0.5` and `12.0`, round them to 2 decimal places, then take the fir three.
>
> Put your answer in a list called `T1_first_three`.

> **Hint.** One line does it: `np.round(np.random.default_rng(7).uniform(0.5, 12.0, 600), 2)[:3]`. Wrap it in `list(...)` so it becomes a plain Python list.

In [12]:
```
# T1 --- same draw as make_delivery_csv, but with seed 7 instead of 42.
T1_first_three = list(np.round(np.random.default_rng(7).uniform(0.5, 12.0

print("T1_first_three =", T1_first_three)
```

```
T1_first_three = [np.float64(7.69), np.float64(10.82), np.float64(9.42)]
```

Task T2 --- Write your own pinned requirements file

Write a file `work/my_requirements.txt` containing exactly three lines,
one each for `numpy`, `pandas` and `scikit-learn`, each pinned to t version actually installed on this machine.

Do not type the version numbers by hand. Ask Python for them, so th file is right on any machine.

> **Hint.** You already saw the pattern in Step 3. `version("numpy")` gives the number; a f-string like `f"numpy=={version('numpy')}"` gives the line. Join three of them witl `"\n"` and use `(WORK / "my_requirements.txt").write_text(...)`.

In [13]:
```
MY_LIBS = ["numpy", "pandas", "scikit-learn"]

# One "name==version" line per library, versions asked of Python, never t
my_lines = [f"{name}=={version(name)}" for name in MY_LIBS]
(WORK / "my_requirements.txt").write_text("\n".join(my_lines) + "\n",
                                          encoding="utf-8")

print(my_lines)
```

```
['numpy==2.5.1', 'pandas==2.3.3', 'scikit-learn==1.9.0']
```

Task T3 --- Fingerprint a run

Write a function `fingerprint(path)` that takes the path of a CSV f and returns a **dictionary** with exactly these three keys:

- `"rows"` --- how many data rows the file has, not counting the header
- `"sha256"` --- the checksum of the file, using the `sha256_of` helper
  from Step 6
- `"seed"` --- the seed the course uses, which is in the variable `SEED`

Then call it on `DATA` and store the result in `T3_fp`.

> **Hint.** `pd.read_csv(path)` gives you a table; `len(...)` on it gives the row count without the header. `sha256_of(path)` is already written for you. Return them as `{"rows": ..., "sha256": ..., "seed": ...}`.

In [14]:
```
def fingerprint(path):
    """Three facts that identify one run: size, contents, and seed."""
```

```
    return {
        "rows": len(pd.read_csv(path)),
        "sha256": sha256_of(path),
        "seed": SEED,
    }


T3_fp = fingerprint(DATA)
print(T3_fp)
```

```
{'rows': 600, 'sha256':
'61bfd2533f407c8ae8a1adcb6e42e1f0fe42cf9c8a6bfe8747673abcafc828c4', 'seed': 4
```

Self-check

Run the cell below to mark your work.

In [15]:

```
# ----------------------------------------------------------------------
# SELF-CHECK --- run this when you have attempted the tasks above.
# It never breaks your notebook. A task you have not done yet simply
# shows FAIL.
# ----------------------------------------------------------------------

_results = []


def _check(label, fn):
    """Evaluate one graded condition without ever raising."""
    try:
        ok = bool(fn())
    except Exception:
        ok = False
    _results.append((label, ok))


_check('T1 | T1_first_three holds three numbers', lambda: len(T1_first_th
_check('T1 | those are the first three distances for seed 7', lambda: all
_check('T2 | work/my_requirements.txt exists', lambda: (WORK / 'my_requir
_check('T2 | it pins all three libraries with ==', lambda: sum(1 for ln i
_check('T3 | fingerprint() returns the three required keys', lambda: set(
_check('T3 | it counts 600 data rows and uses seed 42', lambda: T3_fp['ro
_check('T3 | the checksum matches the file on disk', lambda: T3_fp['sha25

print("======================================================================
print("SELF-CHECK   Practical 01 --- Your MLOps Workbench")
print("======================================================================
for _label, _ok in _results:
    print(f"  [{'PASS' if _ok else 'FAIL'}]  {_label}")
print("----------------------------------------------------------------------
_passed = sum(1 for _, _ok in _results if _ok)
print(f"  {_passed} of {len(_results)} checks passed")
print("======================================================================
if _passed == len(_results):
    print("Well done. Save the notebook and submit it.")
else:
    print("Read the FAIL lines above, fix those tasks, run this cell agai
```

```
================================================================
SELF-CHECK   Practical 01 --- Your MLOps Workbench
================================================================
  [PASS]  T1 | T1_first_three holds three numbers
  [PASS]  T1 | those are the first three distances for seed 7
  [PASS]  T2 | work/my_requirements.txt exists
  [PASS]  T2 | it pins all three libraries with ==
  [PASS]  T3 | fingerprint() returns the three required keys
  [PASS]  T3 | it counts 600 data rows and uses seed 42
  [PASS]  T3 | the checksum matches the file on disk
----------------------------------------------------------------
  7 of 7 checks passed
================================================================
Well done. Save the notebook and submit it.
```

```
---

## What to submit

1. This notebook, with every cell run and its output visible.
```

1. The file `work/my_requirements.txt` that you wrote in Task T2.
2. A screenshot of the output of the last walkthrough cell (`git log`).

Name your file `PXX_<your-roll-number>.ipynb` before you upload it.

How this practical is marked

| What is marked | Marks | |---|---| | Walkthrough run end to end, outputs visible | 3 | | Task T seeds control randomness | 2 | | Task T2 --- a correctly pinned requirements file | 2 | | Task T working fingerprint function | 3 | | **Total** | **10** |

Read more

- Python docs --- Virtual environments and packages --- https://docs.python.org/3/tutorial venv.html
- pip --- Requirements files --- https://pip.pypa.io/en/stable/reference/requirements-file-format/
- NumPy --- Random generator and seeds --- https://numpy.org/doc/stable/reference/random/ generator.html
- Pro Git --- Getting started --- https://git-scm.com/book/en/v2/Getting-Started-First-Time-Git-S

```
# Practical 02 --- Your First Honest Model

**Split the data, train a model, and prove it beats guessing**

SCSE3040 Machine Learning Operations &middot; Bennett University
&middot; Session 2026-27

| | |
|---|---|
| **Lectures this follows** | L03 |
| **Course Outcome** | CO2 |
| **Lab duration** | 120 minutes |
| **Memory needed** | about 300 MB (fine on a 4 GB or 8 GB machine) |
| **Extra software** | nothing beyond the course venv |
| **Marks** | 10 |

---

## Aim

1. Split a dataset into a training part and a testing part, and say
why.
```

1. Train a linear regression model on the delivery data.
2. Measure the error with MAE and RMSE.
3. Compare your model against a baseline, so the score means something.

What you need to know first

Today you build a model that predicts how many minutes a food delivery will take. The idea is sir show a program many past deliveries, and let it work out the pattern.

Two words first.

Features are the facts you know *before* the delivery happens: the distance, how long the restau takes to cook, the traffic, whether it is raining. We call this collection `X`.

Target is the thing you want to predict: the actual minutes taken. We call it `y`.

Now the important part, and the part students get wrong for years. If you train a model on 600 de and then test it on those same 600 deliveries, of course it does well --- it has seen the answers. not a test; it is memory. So we cut the data in two: a **training set** the model learns from, and a **te** never sees until we score it. The test score is the only honest one.

Finally, a score on its own means nothing. Is an error of 4 minutes good? You cannot say until yc what a *stupid* method scores. So we build a **baseline** first: always predict the average delivery ti ignoring every feature. If your clever model cannot beat that, it is not clever.

The measure we use is **MAE** (Mean Absolute Error): on average, how many minutes were we of in minutes, so anyone can understand it.

```
---

## Before you start

- Practical P01 is finished. You know what a seed is and why it
matters.
```

- You can run a notebook cell with Shift + Enter.
- You have seen the delivery dataset and its five columns.

How to run a cell

Click on a grey code cell, then press **Shift + Enter**. The cell runs and the cursor moves to th one. A number appears in the `[ ]` on the left when the cell has finished.

Run the cells in order, from the top. A later cell almost always uses something an earlier c created. If you jump ahead you will see a `NameError`, which just means "you have not ma thing yet".

If everything goes wrong, use the menu: **Kernel -> Restart Kernel and Clear All Outputs**, start again from the first cell. Nothing is damaged by doing this.

In [1]:
```
# Step 0 --- check the workbench before we start.
# This cell only looks; it changes nothing. Run it and read the last line

import sys
from pathlib import Path

print("Python  :", sys.version.split()[0])
print("Folder  :", Path.cwd().name)

_missing = []
for _name in ['numpy', 'pandas', 'sklearn']:
    try:
        __import__(_name)
    except ImportError:
        _missing.append(_name)

for _name in ['numpy', 'pandas', 'sklearn']:
    _mark = "missing" if _name in _missing else "ok"
    print(f"  {_name:<14} {_mark}")

if _missing:
    print()
    print("STOP. Some libraries are missing:", ", ".join(_missing))
    print("Ask your instructor to run the setup in labs/SETUP.md.")
else:
    print()
    print("All good. You can carry on to Step 1.")
```

```
Python  : 3.14.3
Folder  : P02-first-model
  numpy          ok
  pandas         ok
  sklearn        ok

All good. You can carry on to Step 1.
```

Step 0b --- the dataset

Every practical in this course uses the same 600 food deliveries. The next cell makes sure the fil
there.

In [2]:
```python
# The delivery-time dataset every practical in this course uses.
# If the file is missing we build it again from the same seed, so every
# student in the room gets byte-for-byte the same 600 rows.

import csv
from pathlib import Path

import numpy as np

SEED = 42
N_ROWS = 600
DATA = Path("..") / "data" / "delivery_times.csv"


def make_delivery_csv(path=DATA):
    """Write the 600-row delivery dataset. Same formula as the lectures."
    rng = np.random.default_rng(SEED)
    distance_km = np.round(rng.uniform(0.5, 12.0, N_ROWS), 2)
    prep_time_min = np.round(rng.uniform(5, 30, N_ROWS), 0)
    traffic_level = rng.integers(1, 4, N_ROWS)
    rain = rng.binomial(1, 0.25, N_ROWS)
    delivery_min = np.round(
        6.0
        + 3.1 * distance_km
        + 0.65 * prep_time_min
        + 4.2 * traffic_level
        + 5.5 * rain
        + rng.normal(0, 2.5, N_ROWS),
        1,
    )
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(["distance_km", "prep_time_min", "traffic_level",
                    "rain", "delivery_min"])
        for i in range(N_ROWS):
            w.writerow([distance_km[i], int(prep_time_min[i]),
                        int(traffic_level[i]), int(rain[i]), delivery_min
    return path


if not DATA.exists():
    make_delivery_csv()
```

```
    print("dataset rebuilt ->", DATA)
else:
    print("dataset found   ->", DATA)
```

```
dataset found   -> ../data/delivery_times.csv
```

Walkthrough

Read each step, then run its cell.

Step 1 --- Load the deliveries and look at them

Start every project by looking at the data. Never train a model on a table you have not printed.

In [3]:
```
import numpy as np
import pandas as pd

orders = pd.read_csv(DATA)

print("rows, columns:", orders.shape)
print()
print(orders.head())
```

```
rows, columns: (600, 5)

   distance_km  prep_time_min  traffic_level  rain  delivery_min
0         9.40             17              1     0          51.3
1         5.55             24              2     1          54.2
2        10.37             28              3     0          67.7
3         8.52             23              2     0          51.2
4         1.58             29              2     1          42.1
```

Five columns. Four of them are things we know in advance. One of them, `delivery_min`, is t answer.

Step 2 --- Separate the features from the target

`X` holds the four things we know. `y` holds the one thing we want to predict. Capital `X` and sm a convention you will see in every machine learning codebase in the world.

In [4]:
```
FEATURES = ["distance_km", "prep_time_min", "traffic_level", "rain"]
TARGET = "delivery_min"

X = orders[FEATURES]
y = orders[TARGET]

print("X shape:", X.shape, "  <- 600 orders, 4 features each")
print("y shape:", y.shape, "     <- 600 answers")
print()
print(X.head(3))
print()
print(y.head(3))
```

```
X shape: (600, 4)   <- 600 orders, 4 features each
y shape: (600,)      <- 600 answers

   distance_km  prep_time_min  traffic_level  rain
0         9.40             17              1     0
1         5.55             24              2     1
2        10.37             28              3     0

0    51.3
1    54.2
2    67.7
Name: delivery_min, dtype: float64
```

Step 3 --- Cut the data in two

80% to learn from, 20% held back for the test. `random_state=42` is the seed from P01: it ma split identical for everybody in this room, so we can compare our numbers.

In [5]:
```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("training on:", len(X_train), "orders")
print("testing on :", len(X_test), "orders")
print("total      :", len(X_train) + len(X_test))
```

```
training on: 480 orders
testing on : 120 orders
total      : 600
```

From here on, **the model never sees `X_test` or `y_test` until we score it.** Treat those two sealed exam paper.

Step 4 --- The number to beat: always guess the average

Before any model, build the dumbest possible predictor. It ignores distance, traffic, everything, a always answers with the average delivery time from the training data.

Whatever this scores is the bar your model has to clear.

In [6]:
```python
from sklearn.metrics import mean_absolute_error, mean_squared_error

average_time = y_train.mean()
print(f"average delivery time in training data: {average_time:.1f} min")

baseline_guesses = np.full(len(y_test), average_time)
baseline_mae = mean_absolute_error(y_test, baseline_guesses)

print(f"BASELINE MAE: {baseline_mae:.2f} minutes")
print()
print("Meaning: guessing the average is wrong by about")
print(f"{baseline_mae:.0f} minutes on a typical order.")
```

```
average delivery time in training data: 47.1 min
BASELINE MAE: 10.32 minutes

Meaning: guessing the average is wrong by about
10 minutes on a typical order.
```

Step 5 --- Train a real model

Linear regression looks for a straight-line relationship: each extra kilometre adds so many minu adds so many more, and so on. It is the simplest useful model there is, which makes it the right f to try.

Two lines. `.fit()` is the learning.

In [7]:
```python
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

print("Model trained on", len(X_train), "orders.")
print("It has learned", len(model.coef_), "numbers, one per feature.")
```

```
Model trained on 480 orders.
It has learned 4 numbers, one per feature.
```

Step 6 --- Score it on the data it has never seen

Now open the sealed exam paper. We report two numbers:

- **MAE** (Mean Absolute Error): the average miss, in minutes.
- **RMSE** (Root Mean Squared Error): similar, but it punishes big misses much harder. If RMSE above MAE, you have a few terrible predictions hiding among good ones.

In [8]:
```python
predictions = model.predict(X_test)

mae = mean_absolute_error(y_test, predictions)
rmse = float(np.sqrt(mean_squared_error(y_test, predictions)))

print(f"BASELINE MAE : {baseline_mae:6.2f} minutes")
print(f"MODEL    MAE : {mae:6.2f} minutes")
print(f"MODEL   RMSE : {rmse:6.2f} minutes")
print()
improvement = 100 * (baseline_mae - mae) / baseline_mae
print(f"The model is {improvement:.0f}% better than guessing.")
```

```
BASELINE MAE :  10.32 minutes
MODEL    MAE :   1.92 minutes
MODEL   RMSE :   2.48 minutes

The model is 81% better than guessing.
```

That is a result you can report. Not "MAE is 2", but "MAE is 2, against a baseline of 9, on data th had never seen".

Step 7 --- Ask the model what it learned

Linear regression is one of the few models you can read. Each feature gets one number, called a **coefficient**: how many minutes that feature adds per unit.

In [9]:
```
learned = pd.DataFrame({
    "feature": FEATURES,
    "minutes added per unit": model.coef_.round(2),
})

print(learned.to_string(index=False))
print()
print(f"starting point (intercept): {model.intercept_:.1f} minutes")
```

```
     feature  minutes added per unit
  distance_km                    3.07
prep_time_min                    0.65
traffic_level                    4.13
         rain                    5.55

starting point (intercept): 6.4 minutes
```

Read it in plain English: every extra kilometre adds about 3 minutes, rain adds about 5, and eacl in traffic adds about 4. Those numbers should match your everyday experience. If a coefficient lo absurd, that is a bug, not a discovery.

Step 8 --- Predict one new order

Finally, use it the way an app would: one order in, one number out.

A 5 km delivery, 20 minutes of cooking, medium traffic, no rain.

In [10]:
```
new_order = pd.DataFrame([{
    "distance_km": 5.0,
    "prep_time_min": 20,
    "traffic_level": 2,
    "rain": 0,
}])

minutes = model.predict(new_order)[0]
print(f"Predicted delivery time: {minutes:.1f} minutes")
```

```
Predicted delivery time: 43.0 minutes
```

Keep this cell in mind. In Practical P07 this exact call goes behind a web address, so a phone ap ask for it.

Your turn

The walkthrough above is finished. Now you write some code.

There are **3 tasks**. Each one is small. Each one has a hint. Do them in order.

Where you see `# TODO`, replace that line with your own code. Do not delete the variable name left of the `=` sign --- the self-check at the bottom looks for exactly that name.

When you have tried all three, run the **self-check** cell at the end. It prints a table telling you whic are correct. You can run it as many times as you like.

Task T1 --- Work out the baseline yourself

```
Step 4 used the **mean** (the average) as the baseline. Another com
baseline is the **median** --- the middle value.

Build a median baseline: predict `y_train.median()` for every order
the test set, and put its MAE in a variable called `T1_median_mae`.

Then answer, in the variable `T1_which_is_better`, the string
`"mean"` or `"median"` --- whichever gives the *lower* MAE.
```

> **Hint.** Copy Step 4 and change two things: `y_train.median()` instead of `y_train.mean()`, and a new variable name. Compare your number with `baseline_mae` to answer the second part.

In [11]:
```
# T1 --- the median baseline: predict the middle value for every test ora
median_time = y_train.median()
median_guesses = np.full(len(y_test), median_time)
T1_median_mae = mean_absolute_error(y_test, median_guesses)

# Whichever baseline has the lower MAE is the better one.
T1_which_is_better = "mean" if baseline_mae < T1_median_mae else "median"

print("median baseline MAE:", T1_median_mae)
print("better baseline    :", T1_which_is_better)
```

```
median baseline MAE: 10.331666666666669
better baseline    : mean
```

Task T2 --- Does the split change the answer?

```
Everything so far used an 80/20 split with seed 42. Redo the whole
thing with a **70/30 split** and seed **7**:

1. split again into `X_tr2, X_te2, y_tr2, y_te2`
2. train a **new** `LinearRegression` on the new training data
3. put its test MAE in `T2_mae`

This tells you whether your result was real or a lucky split.
```

> **Hint.** `train_test_split(X, y, test_size=0.3, random_state=7)`. Then `model2 = LinearRegression().fit(X_tr2, y_tr2)`. Remember to call `.fit()` -- a fresh model knows nothing until you do.

In [12]:
```
# T2 --- a different split, to see whether the result was real or lucky.
X_tr2, X_te2, y_tr2, y_te2 = train_test_split(
```

```
    X, y, test_size=0.3, random_state=7
)

model2 = LinearRegression().fit(X_tr2, y_tr2)

T2_mae = mean_absolute_error(y_te2, model2.predict(X_te2))

print("70/30 split, seed 7 -> MAE", T2_mae)
```

```
70/30 split, seed 7 -> MAE 2.0963348724086432
```

Task T3 --- One order in, one number out

Write a function `predict_minutes(distance_km, prep_time_min, traffic_level, rain)` that returns the predicted delivery time as a plain number, rounded to one decimal place.

It should use the `model` you trained in Step 5.

Then call it for a 3 km order, 15 minutes of cooking, traffic level in the rain, and store the result in `T3_rainy_order`.

> **Hint.** Inside the function, build a one-row DataFrame exactly like Step 8 did, call `model.predict(...)`, take element `[0]`, and wrap it in `round(..., 1)`. Retur that.

In [13]:
```
def predict_minutes(distance_km, prep_time_min, traffic_level, rain):
    """One order in, one number out."""
    one_order = pd.DataFrame([{
        "distance_km": distance_km,
        "prep_time_min": prep_time_min,
        "traffic_level": traffic_level,
        "rain": rain,
    }])
    return round(float(model.predict(one_order)[0]), 1)


# 3 km, 15 minutes prep, traffic level 1, raining
T3_rainy_order = predict_minutes(3.0, 15, 1, 1)

print("rainy 3 km order ->", T3_rainy_order, "minutes")
```

```
rainy 3 km order -> 35.0 minutes
```

Self-check

Run the cell below to mark your work.

In [14]:
```
# ---------------------------------------------------------------------
# SELF-CHECK --- run this when you have attempted the tasks above.
# It never breaks your notebook. A task you have not done yet simply
# shows FAIL.
# ---------------------------------------------------------------------
```

```
_results = []


def _check(label, fn):
    """Evaluate one graded condition without ever raising."""
    try:
        ok = bool(fn())
    except Exception:
        ok = False
    _results.append((label, ok))


_check('T1 | T1_median_mae is the MAE of a median baseline', lambda: abs(
_check('T1 | T1_which_is_better names the lower-MAE baseline', lambda: T1
_check('T2 | the new test set holds 30% of the orders', lambda: len(X_te2
_check('T2 | model2 is a trained LinearRegression', lambda: hasattr(model
_check("T2 | T2_mae is that model's MAE on the new test set", lambda: abs
_check('T3 | predict_minutes returns a single number', lambda: isinstance
_check('T3 | it agrees with the trained model', lambda: abs(predict_minut
_check('T3 | rain makes the same order take longer', lambda: predict_minu
_check('T3 | T3_rainy_order is the rainy 3 km prediction', lambda: abs(fl

print("======================================================================")
print("SELF-CHECK   Practical 02 --- Your First Honest Model")
print("======================================================================")
for _label, _ok in _results:
    print(f"  [{'PASS' if _ok else 'FAIL'}]  {_label}")
print("----------------------------------------------------------------------")
_passed = sum(1 for _, _ok in _results if _ok)
print(f"  {_passed} of {len(_results)} checks passed")
print("======================================================================")
if _passed == len(_results):
    print("Well done. Save the notebook and submit it.")
else:
    print("Read the FAIL lines above, fix those tasks, run this cell agai
```

```
======================================================================
SELF-CHECK   Practical 02 --- Your First Honest Model
======================================================================
  [PASS]  T1 | T1_median_mae is the MAE of a median baseline
  [PASS]  T1 | T1_which_is_better names the lower-MAE baseline
  [PASS]  T2 | the new test set holds 30% of the orders
  [PASS]  T2 | model2 is a trained LinearRegression
  [PASS]  T2 | T2_mae is that model's MAE on the new test set
  [PASS]  T3 | predict_minutes returns a single number
  [PASS]  T3 | it agrees with the trained model
  [PASS]  T3 | rain makes the same order take longer
  [PASS]  T3 | T3_rainy_order is the rainy 3 km prediction
----------------------------------------------------------------------
  9 of 9 checks passed
======================================================================
Well done. Save the notebook and submit it.
```

What to submit

1. This notebook, with every cell run and its output visible.

1. In a markdown cell at the end, one sentence saying whether your model beat the baseline a how much.

Name your file `PXX_<your-roll-number>.ipynb` before you upload it.

How this practical is marked

| What is marked | Marks | |---|---| | Walkthrough run end to end, outputs visible | 3 | | Task T baseline computed correctly | 2 | | Task T2 --- model retrained on a different split | 2 | | Task working single-order predictor | 3 | | **Total** | **10** |

Read more

- scikit-learn --- train_test_split --- https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

- scikit-learn --- Linear regression --- https://scikit-learn.org/stable/modules/linear_model.html
- scikit-learn --- Metrics for regression --- https://scikit-learn.org/stable/modules/model_evalua

---

My result

My linear regression model beat the mean baseline by a wide margin: MAE dropped from **10.32** (always guessing the average) to **1.92 minutes** on the held-out test set, an 81% reduction in err

```
# Practical 03 --- Choosing a Model Honestly

**Overfitting, the train/test gap, and cross-validation**

SCSE3040 Machine Learning Operations &middot; Bennett
University &middot; Session 2026-27

| | |
|---|---|
| **Lectures this follows** | L04–L05 |
| **Course Outcome** | CO2 |
| **Lab duration** | 120 minutes |
| **Memory needed** | about 400 MB (fine on a 4 GB or 8 GB
machine) |
| **Extra software** | nothing beyond the course venv |
| **Marks** | 10 |

---

## Aim

1. Train three different models on the same data and compare
them fairly.
```

2. See overfitting happen, and measure it as a gap between two scores.
3. Use cross-validation instead of trusting one lucky split.
4. Choose a model for a reason you can defend, not because it scored best.

What you need to know first

In P02 you trained one model and it did well. A reasonable next thought is "let me try a more powerful model and do even better". Today you find out why that thought is dangerous.

A **decision tree** is a model that asks yes/no questions: *is the distance more than 6 km? is it raining?* Ask enough questions and you can separate every single training order from every other one. At that point the model has not learned a pattern --- it has memorised the training data. It will look perfect in training and fall apart on anything new.

That failure has a name: **overfitting**. And it has a measurement. Score the model twice, once on the data it trained on and once on data it has never seen. The distance between those two numbers is the **train/test gap**. A small gap means the model learned something general. A big gap means it memorised.

There is one more problem to fix. In P02 your test set was one particular 120 orders, chosen by one particular seed. What if those 120 happened to be easy? **Cross-validation** removes that luck: it cuts the data into 5 equal parts, trains 5 times, and each time tests on a different part. You get 5 scores instead of 1, and you report the average. It costs 5 times the computing time, which on this dataset means well under a second.

The habit to build today: *the best score is not the best model.* A simple model that scores almost as well is usually the one to ship, because you can explain it, it trains faster, and it breaks less.

```
---

## Before you start

- Practical P02 is finished. You can split data, train a
model and read an MAE.
```

- You remember that the test set is the only honest score.

How to run a cell

Click on a grey code cell, then press **Shift + Enter**. The cell runs and the cursor moves to the next one. A number appears in the `[ ]` on the left when the cell has finished.

Run the cells in order, from the top. A later cell almost always uses something an earlier cell created. If you jump ahead you will see a `NameError`, which just means "you have not made that thing yet".

If everything goes wrong, use the menu: **Kernel -> Restart Kernel and Clear All Outputs**, then start again from the first cell. Nothing is damaged by doing this.

In [1]:
```
# Step 0 --- check the workbench before we start.
# This cell only looks; it changes nothing. Run it and read the last line.

import sys
from pathlib import Path

print("Python  :", sys.version.split()[0])
print("Folder  :", Path.cwd().name)

_missing = []
for _name in ['numpy', 'pandas', 'sklearn', 'matplotlib']:
    try:
        __import__(_name)
    except ImportError:
        _missing.append(_name)
```

```
for _name in ['numpy', 'pandas', 'sklearn', 'matplotlib']:
    _mark = "missing" if _name in _missing else "ok"
    print(f"  {_name:<14} {_mark}")

if _missing:
    print()
    print("STOP. Some libraries are missing:", ", ".join(_missing))
    print("Ask your instructor to run the setup in labs/SETUP.md.")
else:
    print()
    print("All good. You can carry on to Step 1.")
```

```
Python  : 3.14.3
Folder  : P03-model-choice
  numpy          ok
  pandas         ok
  sklearn        ok
  matplotlib     ok

All good. You can carry on to Step 1.
```

Step 0b --- the dataset

Every practical in this course uses the same 600 food deliveries. The next cell makes sure the file is there.

In [2]:
```
# The delivery-time dataset every practical in this course uses.
# If the file is missing we build it again from the same seed, so every
# student in the room gets byte-for-byte the same 600 rows.

import csv
from pathlib import Path

import numpy as np

SEED = 42
N_ROWS = 600
DATA = Path("..") / "data" / "delivery_times.csv"


def make_delivery_csv(path=DATA):
    """Write the 600-row delivery dataset. Same formula as the lectures."""
    rng = np.random.default_rng(SEED)
    distance_km = np.round(rng.uniform(0.5, 12.0, N_ROWS), 2)
    prep_time_min = np.round(rng.uniform(5, 30, N_ROWS), 0)
    traffic_level = rng.integers(1, 4, N_ROWS)
    rain = rng.binomial(1, 0.25, N_ROWS)
    delivery_min = np.round(
        6.0
        + 3.1 * distance_km
        + 0.65 * prep_time_min
        + 4.2 * traffic_level
        + 5.5 * rain
```

```
        + rng.normal(0, 2.5, N_ROWS),
        1,
    )
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(["distance_km", "prep_time_min", "traffic_level",
                    "rain", "delivery_min"])
        for i in range(N_ROWS):
            w.writerow([distance_km[i], int(prep_time_min[i]),
                        int(traffic_level[i]), int(rain[i]), delivery_min[i]])
    return path


if not DATA.exists():
    make_delivery_csv()
    print("dataset rebuilt ->", DATA)
else:
    print("dataset found   ->", DATA)
```

```
dataset found   -> ../data/delivery_times.csv
```

Walkthrough

Read each step, then run its cell.

Step 1 --- Set up the data once, for all three models

Same data, same split, same seed as P02. Every model today gets exactly the same deal --- otherwise the comparison is worthless.

In [3]:
```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

orders = pd.read_csv(DATA)

FEATURES = ["distance_km", "prep_time_min", "traffic_level", "rain"]
X = orders[FEATURES]
y = orders["delivery_min"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("train:", len(X_train), " test:", len(X_test))
```

```
train: 480  test: 120
```

Step 2 --- A helper that scores a model both ways

We will do the same two measurements many times today, so write them once. This function returns the training MAE, the test MAE, and the gap between them.

In [4]:
```
from sklearn.metrics import mean_absolute_error

def score_both_ways(model, name):
    """Train the model, then score it on train and on test."""
    model.fit(X_train, y_train)
    train_mae = mean_absolute_error(y_train, model.predict(X_train))
    test_mae = mean_absolute_error(y_test, model.predict(X_test))
    gap = test_mae - train_mae
    print(f"{name:<26} train {train_mae:5.2f}   "
          f"test {test_mae:5.2f}   gap {gap:5.2f}")
    return {"name": name, "train": train_mae,
            "test": test_mae, "gap": gap}

print("helper ready")
```

```
helper ready
```

Step 3 --- Model 1: linear regression, the honest baseline

The model from P02. Watch the gap: it should be tiny, because a straight line is too simple to memorise anything.

In [5]:
```
from sklearn.linear_model import LinearRegression

linear = score_both_ways(LinearRegression(), "LinearRegression")
```

```
LinearRegression           train  2.04   test  1.92   gap -0.11
```

Train and test scores almost equal. This model is not overfitting. It may not be the most accurate thing possible, but it is telling you the truth.

Step 4 --- Model 2: a decision tree with no limits

Now the dangerous one. A decision tree with no maximum depth is allowed to keep asking questions until every training order sits in its own little box.

Look at the training score first. Then look at the test score.

In [6]:
```
from sklearn.tree import DecisionTreeRegressor

wild_tree = score_both_ways(
    DecisionTreeRegressor(random_state=42), "DecisionTree (no limit)"
)
```

```
DecisionTree (no limit)    train  0.00   test  3.43   gap  3.43
```

A training MAE near **zero** and a much worse test MAE. The model got every training order perfectly right and learned nothing useful.

That gap is overfitting. You have now seen it happen, not just been told about it.

Step 5 --- Model 3: the same tree, but shallow

Now stop the tree early. `max_depth=4` means it may only ask four questions before it has to answer. It can no longer memorise.

In [7]:
```
small_tree = score_both_ways(
    DecisionTreeRegressor(max_depth=4, random_state=42),
    "DecisionTree (depth 4)",
)
```

```
DecisionTree (depth 4)     train  3.66   test  4.23   gap  0.57
```

Worse on training, better on test, and a far smaller gap. Limiting a model's freedom made it *more* useful. That idea is the whole of model selection.

Step 6 --- Model 4: a random forest

A **random forest** grows many different trees and averages their answers. Individual trees still overfit, but their mistakes point in different directions and largely cancel out.

We use 50 trees. More would be slower and no better here.

In [8]:
```
from sklearn.ensemble import RandomForestRegressor

forest = score_both_ways(
    RandomForestRegressor(n_estimators=50, random_state=42),
    "RandomForest (50 trees)",
)
```

```
RandomForest (50 trees)    train  0.99   test  2.40   gap  1.40
```

Step 7 --- Put the four results side by side

Never compare models from memory. Put them in a table.

In [9]:
```
results = pd.DataFrame([linear, wild_tree, small_tree, forest])
results = results.round(2).sort_values("test")

print(results.to_string(index=False))
print()
best = results.iloc[0]
print(f"Lowest test MAE: {best['name']} at {best['test']:.2f} minutes")
```

```
                   name  train  test   gap
       LinearRegression   2.04  1.92 -0.11
RandomForest (50 trees)   0.99  2.40  1.40
DecisionTree (no limit)   0.00  3.43  3.43
 DecisionTree (depth 4)   3.66  4.23  0.57

Lowest test MAE: LinearRegression at 1.92 minutes
```

Read the `gap` column as carefully as the `test` column. A model with a good test score and a huge gap got lucky, and luck does not survive contact with real users.

Step 8 --- Draw the gap, because a table makes you read

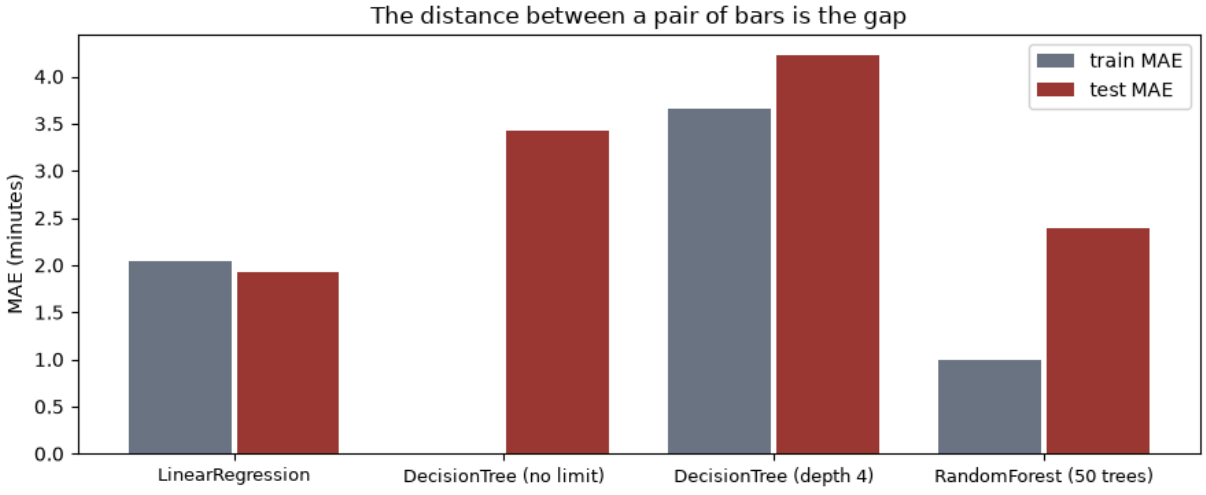
The table is correct but slow --- you must read eight numbers and subtract in your head. A chart does the subtraction for your eyes.

matplotlib is Python's standard charting library. It has been sitting in your course environment since P01; today it earns its place. The first line, `%matplotlib inline`, tells Jupyter to draw the picture right below the cell.

In [10]:
```
%matplotlib inline
import matplotlib.pyplot as plt

four = [linear, wild_tree, small_tree, forest]
xs = np.arange(len(four))

fig, ax = plt.subplots(figsize=(9, 3.8))
ax.bar(xs - 0.2, [m["train"] for m in four], 0.38,
       label="train MAE", color="#6B7785")
ax.bar(xs + 0.2, [m["test"] for m in four], 0.38,
       label="test MAE", color="#9E3B33")
ax.set_xticks(xs, [m["name"] for m in four], fontsize=9)
ax.set_ylabel("MAE (minutes)")
ax.set_title("The distance between a pair of bars is the gap")
ax.legend()
plt.tight_layout()
plt.show()
```



Look at the pair for the no-limit tree. Its grey train bar has **vanished** --- an MAE of 0.00 draws as no bar at all --- while its red test bar is among the tallest. Bars far apart mean the model memorised; bars close together mean it learned.

The linear model's two bars are the closest pair on the chart. That is what honesty looks like.

Step 9 --- Stop trusting one split: cross-validation

Every number above came from one 120-order test set. Change the seed and they all move a little.

5-fold cross-validation cuts the data into 5 parts and trains 5 times, testing on a different part each time. We report the average and the spread.

scikit-learn reports errors as *negative* numbers, because its convention is that bigger is better. We flip the sign back.

In [11]:
```
from sklearn.model_selection import cross_val_score

def cross_validate(model, name):
    """Average MAE over 5 different train/test splits."""
    scores = -cross_val_score(
        model, X, y, cv=5, scoring="neg_mean_absolute_error"
    )
    print(f"{name:<26} MAE {scores.mean():5.2f} "
          f"(+/- {scores.std():4.2f})")
    return scores.mean()

cv_linear = cross_validate(LinearRegression(), "LinearRegression")
cv_tree = cross_validate(
    DecisionTreeRegressor(max_depth=4, random_state=42),
    "DecisionTree (depth 4)")
cv_forest = cross_validate(
    RandomForestRegressor(n_estimators=50, random_state=42),
    "RandomForest (50 trees)")
```

```
LinearRegression           MAE  2.03 (+/- 0.16)
DecisionTree (depth 4)     MAE  4.57 (+/- 0.18)
RandomForest (50 trees)    MAE  2.69 (+/- 0.40)
```

The `+/-` is the spread across the 5 folds. If two models are closer together than their spread, you cannot honestly say one is better.

Step 10 --- Tune one setting, properly

How deep should the tree be? Do not guess, and do not try depths on the test set --- that would leak the answers.

Try each depth with cross-validation, which never touches the test set at all.

In [12]:
```
depths = [2, 3, 4, 6, 8, None]
scores_by_depth = {}

for depth in depths:
    tree = DecisionTreeRegressor(max_depth=depth, random_state=42)
    mae = -cross_val_score(
        tree, X, y, cv=5, scoring="neg_mean_absolute_error"
    ).mean()
    scores_by_depth[depth] = round(mae, 3)
    print(f"max_depth={str(depth):<5} MAE {mae:5.2f}")

best_depth = min(scores_by_depth, key=scores_by_depth.get)
print()
print("best depth:", best_depth)
```

```
max_depth=2     MAE  5.86
max_depth=3     MAE  4.85
max_depth=4     MAE  4.57
max_depth=6     MAE  3.64
max_depth=8     MAE  3.39
max_depth=None  MAE  3.47

best depth: 8
```

Read the **shape**, not just the winner.

Depth 2 is the worst of all: too simple to capture anything, which is called **underfitting**. The scores improve as the tree is allowed more room, reach their best in the middle, and then get slightly worse again when the limit is removed altogether.

Too rigid and too free are both bad. The good answer is in between, and the only way to find it is to measure --- which is what you just did, without ever touching the test set.

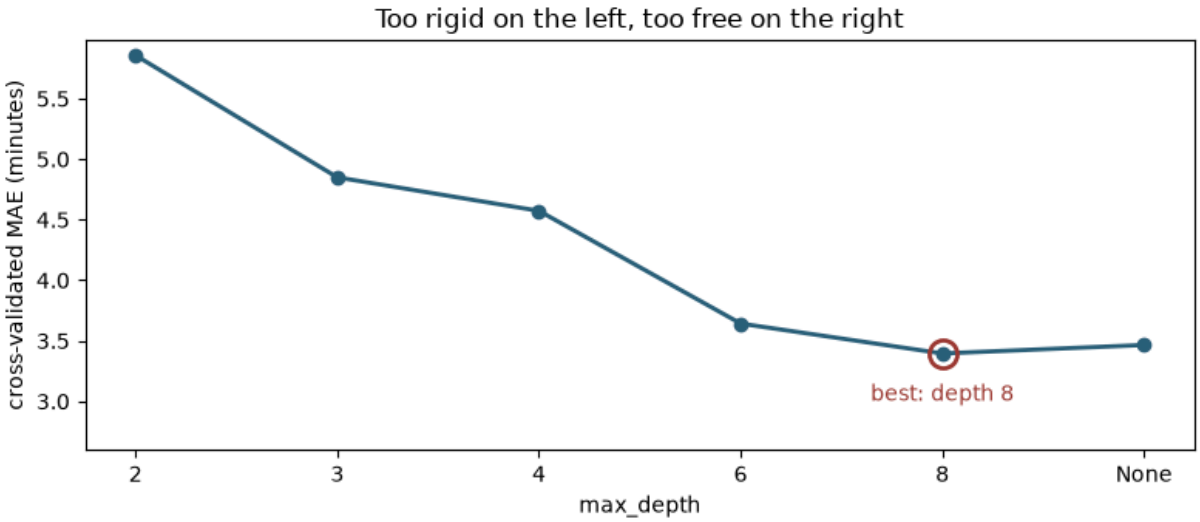
Step 11 --- Draw the sweep, because ten numbers hide a shape

The loop printed six scores. Their *shape* is the real lesson, and a shape is easier to see than to read. Draw MAE against depth.

The red circle marks the depth cross-validation liked best.

In [13]:
```
xs = np.arange(len(depths))
maes = [scores_by_depth[d] for d in depths]
best_i = int(np.argmin(maes))
```

```
fig, ax = plt.subplots(figsize=(8, 3.6))
ax.plot(xs, maes, marker="o", color="#2C5F79", linewidth=2)
ax.plot(xs[best_i], maes[best_i], marker="o", markersize=13,
        markerfacecolor="none", markeredgecolor="#9E3B33",
        markeredgewidth=2)
ax.annotate(f"best: depth {depths[best_i]}",
            (xs[best_i], maes[best_i]),
            xytext=(0, -22), textcoords="offset points",
            ha="center", color="#9E3B33")
ax.set_xticks(xs, [str(d) for d in depths])
ax.set_ylim(bottom=min(maes) - 0.8)
ax.set_xlabel("max_depth")
ax.set_ylabel("cross-validated MAE (minutes)")
ax.set_title("Too rigid on the left, too free on the right")
plt.tight_layout()
plt.show()
```



Now the shape is unmissable. The left end towers over everything: depth 2 is the worst score on the whole chart --- **underfitting**, a model too rigid to learn anything. The curve falls as the tree earns more freedom, bottoms out in the middle, then ticks back up when the last limit is removed.

Both ends of the curve are bad. The answer lives at the bottom, and you found it without ever touching the test set.

Step 12 --- Choose, and be able to say why

Now make the decision the way a professional would, out loud.

In [14]:
```
print("Cross-validated MAE, best first:")
for name, mae in sorted(
    {"LinearRegression": cv_linear,
     "DecisionTree(4)": cv_tree,
     "RandomForest(50)": cv_forest}.items(),
    key=lambda kv: kv[1],
):
```

```
    print(f"  {name:<20} {mae:5.2f}")

print()
print("Our choice for the delivery service: LinearRegression.")
print("Reasons:")
print("  1. It has the lowest cross-validated error here.")
print("  2. It trains in milliseconds and needs almost no memory.")
print("  3. We can read its coefficients and explain a prediction")
print("     to a restaurant owner. Nobody can read 50 trees.")
print()
print("Be careful with reason 1. This dataset really was built")
print("from a straight line, so a straight line wins. On messier")
print("real data the forest usually wins instead -- and then you")
print("must decide what its extra accuracy is worth, against")
print("reasons 2 and 3, which do not change.")
```

```
Cross-validated MAE, best first:
  LinearRegression      2.03
  RandomForest(50)      2.69
  DecisionTree(4)       4.57

Our choice for the delivery service: LinearRegression.
Reasons:
  1. It has the lowest cross-validated error here.
  2. It trains in milliseconds and needs almost no memory.
  3. We can read its coefficients and explain a prediction
     to a restaurant owner. Nobody can read 50 trees.

Be careful with reason 1. This dataset really was built
from a straight line, so a straight line wins. On messier
real data the forest usually wins instead -- and then you
must decide what its extra accuracy is worth, against
reasons 2 and 3, which do not change.
```

You will not always choose the simple model. But you must always be able to say why you chose what you chose. "It scored best" is not a reason a senior engineer will accept on its own --- and, as the last paragraph says, it will not always be true either.

Your turn

The walkthrough above is finished. Now you write some code.

There are **3 tasks**. Each one is small. Each one has a hint. Do them in order.

Where you see `# TODO`, replace that line with your own code. Do not delete the variable name on the left of the `=` sign --- the self-check at the bottom looks for exactly that name.

When you have tried all three, run the **self-check** cell at the end. It prints a table telling you which tasks are correct. You can run it as many times as you like.

Task T1 --- Measure overfitting yourself

```
Train a `DecisionTreeRegressor` with `max_depth=12` and
`random_state=42`, and work out its train/test gap by hand.

Store the three numbers in `T1_train_mae`, `T1_test_mae` and
`T1_gap`.

`T1_gap` must be the test MAE **minus** the train MAE.
```

> **Hint.** You may not use `score_both_ways` for this one -- write the three lines out, so you can see what the helper was doing. Fit the tree, then call `mean_absolute_error` twice: once with `y_train` and `X_train`, once with `y_test` and `X_test`.

In [15]:
```
deep_tree = DecisionTreeRegressor(max_depth=12, random_state=42)

# T1 --- written out longhand, so the gap is visible.
deep_tree.fit(X_train, y_train)
T1_train_mae = mean_absolute_error(y_train, deep_tree.predict(X_train))
T1_test_mae = mean_absolute_error(y_test, deep_tree.predict(X_test))
T1_gap = T1_test_mae - T1_train_mae

print(f"train {T1_train_mae}  test {T1_test_mae}  gap {T1_gap}")
```

```
train 0.04695601851851851  test 3.397986111111111  gap 3.3510300925925924
```

Task T2 --- Tune the forest, without touching the test set

```
A random forest has a setting `max_depth` too. Use **5-fold
cross-validation** to find which of `[2, 4, 6, None]` is best
for a
```

`RandomForestRegressor(n_estimators=50, random_state=42)`.

Build a dictionary `T2_scores` mapping each depth to its cross-validated MAE, then put the winning depth in `T2_best_depth`.

You must not use `X_test` or `y_test` anywhere in this task.

> **Hint.** Step 10 does exactly this for a single tree. Copy that loop and change two things: the model, and the list of depths. The winner is `min(T2_scores, key=T2_scores.get)`.

In [16]:
```
T2_depths = [2, 4, 6, None]

# T2 --- tuning on cross-validation only. The test set is not touched here.
T2_scores = {}
for depth in T2_depths:
    forest_at_depth = RandomForestRegressor(
        n_estimators=50, max_depth=depth, random_state=42
    )
    mae = -cross_val_score(
        forest_at_depth, X, y, cv=5, scoring="neg_mean_absolute_error"
    ).mean()
    T2_scores[depth] = round(mae, 3)

T2_best_depth = min(T2_scores, key=T2_scores.get)

print(T2_scores)
print("best:", T2_best_depth)
```

```
{2: np.float64(5.271), 4: np.float64(3.643), 6: np.float64(2.844), None: np.
float64(2.689)}
best: None
```

Task T3 --- A comparison you can reuse

Write a function `compare(models)` that takes a **dictionary** of `{name: model}` and returns a dictionary of `{name: test MAE}`.

It must fit each model on `X_train`/`y_train` and score it on `X_test`/`y_test`.

Then call it on the three models given in the stub and store the answer in `T3_table`.

> **Hint.** Loop over `models.items()`, call `.fit(...)`, then `mean_absolute_error(y_test, m.predict(X_test))`, and collect the results into a new dictionary that you `return` at the end.

In [17]:
```
def compare(models):
    """Fit each model on the training data, score it on the test set."""
    return {
        name: mean_absolute_error(y_test, m.fit(X_train, y_train).predict(X_
        for name, m in models.items()
    }


T3_table = compare({
    "linear": LinearRegression(),
    "tree4": DecisionTreeRegressor(max_depth=4, random_state=42),
    "forest": RandomForestRegressor(n_estimators=50, random_state=42),
})

print(T3_table)
```

```
{'linear': 1.9246530798338346, 'tree4': 4.228332168713071, 'forest': 2.39888
333333333}
```

Self-check

Run the cell below to mark your work.

In [18]:
```
# ---------------------------------------------------------------------
# SELF-CHECK --- run this when you have attempted the tasks above.
# It never breaks your notebook. A task you have not done yet simply
# shows FAIL.
# ---------------------------------------------------------------------

_results = []


def _check(label, fn):
    """Evaluate one graded condition without ever raising."""
    try:
        ok = bool(fn())
    except Exception:
        ok = False
    _results.append((label, ok))


_check('T1 | the depth-12 tree is trained', lambda: hasattr(deep_tree, 'tree
_check("T1 | T1_train_mae and T1_test_mae are that tree's two scores", lambd
_check('T1 | T1_gap is test minus train, and it is positive', lambda: abs(fl
_check('T2 | T2_scores has one score per depth', lambda: set(T2_scores) == {
_check('T2 | every score is a plausible MAE in minutes', lambda: all(0 < flo
_check('T2 | T2_best_depth is the depth with the lowest score', lambda: T2_b
_check('T3 | compare() returns one score per model', lambda: set(T3_table) =
_check('T3 | the linear score matches the walkthrough', lambda: abs(float(T3

print("=================================================================")
print("SELF-CHECK   Practical 03 --- Choosing a Model Honestly")
```

```
print("======================================================================")
for _label, _ok in _results:
    print(f"  [{'PASS' if _ok else 'FAIL'}]  {_label}")
print("----------------------------------------------------------------------")
_passed = sum(1 for _, _ok in _results if _ok)
print(f"  {_passed} of {len(_results)} checks passed")
print("======================================================================")
if _passed == len(_results):
    print("Well done. Save the notebook and submit it.")
else:
    print("Read the FAIL lines above, fix those tasks, run this cell again.")
```

```
======================================================================
SELF-CHECK   Practical 03 --- Choosing a Model Honestly
======================================================================
  [PASS]  T1 | the depth-12 tree is trained
  [PASS]  T1 | T1_train_mae and T1_test_mae are that tree's two scores
  [PASS]  T1 | T1_gap is test minus train, and it is positive
  [PASS]  T2 | T2_scores has one score per depth
  [PASS]  T2 | every score is a plausible MAE in minutes
  [PASS]  T2 | T2_best_depth is the depth with the lowest score
  [PASS]  T3 | compare() returns one score per model
  [PASS]  T3 | the linear score matches the walkthrough
----------------------------------------------------------------------
  8 of 8 checks passed
======================================================================
Well done. Save the notebook and submit it.
```

What to submit

1. This notebook, with every cell run and its output visible.
2. In a markdown cell at the end, three sentences: which model you would put in front of real customers, why, and what you gave up by choosing it.

Name your file `PXX_<your-roll-number>.ipynb` before you upload it.

How this practical is marked

| What is marked | Marks | |---|---| | Walkthrough run end to end, outputs visible | 3 | | Task T1 --- overfitting measured as a gap | 2 | | Task T2 --- best setting found by cross-validation | 3 | | Task T3 --- a reusable comparison function | 2 | | **Total** | **10** |

Read more

- scikit-learn --- Underfitting vs overfitting --- https://scikit-learn.org/stable/auto_examples/model_selection/plot_underfitting_overfitting.html

- scikit-learn --- Cross-validation --- https://scikit-learn.org/stable/modules/cross_validation.html
- scikit-learn --- Decision trees --- https://scikit-learn.org/stable/modules/tree.html
- scikit-learn --- Ensemble methods --- https://scikit-learn.org/stable/modules/ensemble.html
- matplotlib --- Quick start guide --- https://matplotlib.org/stable/users/explain/quick_start.html

---

My verdict

I would put the **LinearRegression** model in front of real customers: it had the lowest cross-validated MAE of the three, it trains in milliseconds, and its four coefficients can be read out loud to a restaurant owner as "each extra kilometre costs about three minutes".

What I gave up is flexibility --- a straight line cannot bend, so if delivery time ever stops being roughly linear in these four features (a new city, a curfew, a bicycle fleet) the forest would overtake it and I would have to revisit this choice.

I also gave up the small chance that the forest's extra capacity was picking up a real pattern rather than noise; on this dataset, which really was generated from a straight line, that chance was not worth the loss of explainability.

Practical 04 --- From Notebook to Package

Turn scattered cells into a folder of code that someone else can run

SCSE3040 Machine Learning Operations · Bennett University · Session 2026-27

Lectures this follows	L07
Course Outcome	CO3
Lab duration	120 minutes
Memory needed	about 300 MB (fine on a 4 GB or 8 GB machine)
Extra software	nothing beyond the course venv
Marks	10

Aim

1. Explain why a notebook stops being good enough for real work.
2. Turn cells into functions, and functions into a package of modules.
3. Import your own code and use it, the way a library is used.
4. Run your project from the command line, with no notebook at all.

What you need to know first

Everything you have written so far lives in notebook cells. That is fine for exploring. It stops being moment somebody else needs to run your work.

There are three reasons.

Hidden state. A notebook remembers everything you ran, including cells you have since edited deleted. The screen can show code that never actually ran, next to a result produced by code th longer exists. Nobody can tell by looking.

Order. Notebook cells can be run in any order. Yours works because you happened to run them right sequence today. On another machine, top to bottom, it may not.

Nothing can import a notebook. A web service cannot `import` a cell. A test cannot call a cel scheduled job cannot run a cell. To be used by other software, code must live in a **module**: an o `.py` file.

So today you refactor. **Refactoring** means changing how code is arranged without changing wh does. You will take the working code from P02, put it into functions, group those functions into fil

job, and put those files into a folder with an `__init__.py` in it. A folder like that is called a **pac** and Python can import it.

At the end you will run your whole training pipeline from a terminal, with one command and no n That is the shape every later practical needs: P05 tests this package, P07 serves it, P08 puts it i container.

Before you start

- Practicals P01-P03 are finished. You have trained and scored a model.
- You know what a function is, and what `import` does.

How to run a cell

Click on a grey code cell, then press **Shift + Enter**. The cell runs and the cursor moves to the ne number appears in the `[ ]` on the left when the cell has finished.

Run the cells in order, from the top. A later cell almost always uses something an earlier cell c you jump ahead you will see a `NameError`, which just means "you have not made that thing y

If everything goes wrong, use the menu: **Kernel -> Restart Kernel and Clear All Outputs**, then again from the first cell. Nothing is damaged by doing this.

In [1]:
```
# Step 0 --- check the workbench before we start.
# This cell only looks; it changes nothing. Run it and read the last line

import sys
from pathlib import Path

print("Python  :", sys.version.split()[0])
print("Folder  :", Path.cwd().name)

_missing = []
for _name in ['numpy', 'pandas', 'sklearn', 'joblib']:
    try:
        __import__(_name)
    except ImportError:
        _missing.append(_name)

for _name in ['numpy', 'pandas', 'sklearn', 'joblib']:
    _mark = "missing" if _name in _missing else "ok"
    print(f"  {_name:<14} {_mark}")

if _missing:
    print()
    print("STOP. Some libraries are missing:", ", ".join(_missing))
    print("Ask your instructor to run the setup in labs/SETUP.md.")
else:
    print()
    print("All good. You can carry on to Step 1.")
```

```
Python  : 3.14.7
Folder  : P04-package
```

```
numpy          ok
pandas         ok
sklearn        ok
joblib         ok

All good. You can carry on to Step 1.
```

Step 0b --- the dataset

Every practical in this course uses the same 600 food deliveries. The next cell makes sure the fil
there.

In [2]:

```python
# The delivery-time dataset every practical in this course uses.
# If the file is missing we build it again from the same seed, so every
# student in the room gets byte-for-byte the same 600 rows.

import csv
from pathlib import Path

import numpy as np

SEED = 42
N_ROWS = 600
DATA = Path("..") / "data" / "delivery_times.csv"


def make_delivery_csv(path=DATA):
    """Write the 600-row delivery dataset. Same formula as the lectures."
    rng = np.random.default_rng(SEED)
    distance_km = np.round(rng.uniform(0.5, 12.0, N_ROWS), 2)
    prep_time_min = np.round(rng.uniform(5, 30, N_ROWS), 0)
    traffic_level = rng.integers(1, 4, N_ROWS)
    rain = rng.binomial(1, 0.25, N_ROWS)
    delivery_min = np.round(
        6.0
        + 3.1 * distance_km
        + 0.65 * prep_time_min
        + 4.2 * traffic_level
        + 5.5 * rain
        + rng.normal(0, 2.5, N_ROWS),
        1,
    )
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(["distance_km", "prep_time_min", "traffic_level",
                    "rain", "delivery_min"])
        for i in range(N_ROWS):
            w.writerow([distance_km[i], int(prep_time_min[i]),
                        int(traffic_level[i]), int(rain[i]), delivery_min
    return path


if not DATA.exists():
    make_delivery_csv()
    print("dataset rebuilt ->", DATA)
```

```
else:
    print("dataset found   ->", DATA)
```

```
dataset found   -> ../data/delivery_times.csv
```

Walkthrough

Read each step, then run its cell.

Step 1 --- See the problem before fixing it

A quick demonstration of hidden state. Watch carefully.

In [3]:
```
total = 10
total = total + 5
print("total is", total)
```

```
total is 15
```

Now imagine you go back, change the first line to `total = 100`, and run **only** the second line
The screen would show `total = 100` at the top and `total is 20` at the bottom. Both are
screen. Only one of them is true.

That is not a silly mistake. It is the normal way notebooks fail, and it is why production code does
in them.

Step 2 --- Plan the package before writing it

Group code by **job**, not by the order you happened to write it. Our project has three jobs, so it ge
modules.

```
work/
  delivery/
    __init__.py     marks this folder as a package
    data.py         load the CSV, split it
    features.py     check and describe a single order
    model.py        train, score, save, load
  train.py          the script a human runs
```

One rule: **a module should be describable in one sentence.** If you cannot, it is doing two jobs
wants splitting.

In [4]:
```
from pathlib import Path

WORK = Path("work")
PKG = WORK / "delivery"
PKG.mkdir(parents=True, exist_ok=True)

print("package folder ready:", PKG.resolve())
```

```
package folder ready: /Users/archit/Desktop/mlops-lab/SCSE3040-Lab/P04-packag
work/delivery
```

Step 3 --- Write the first module: data.py

This module does one thing: get the delivery data, split into training and testing.

We write the file from the notebook using `write_text`. In real work you would type it in an ed we do it this way so the whole practical stays in one place.

In [5]:
```
data_py = '''"""Loading and splitting the delivery dataset."""

from pathlib import Path

import pandas as pd
from sklearn.model_selection import train_test_split

FEATURES = ["distance_km", "prep_time_min", "traffic_level", "rain"]
TARGET = "delivery_min"
SEED = 42


def load_orders(path):
    """Read the delivery CSV into a table."""
    return pd.read_csv(Path(path))


def split_orders(orders, test_size=0.2):
    """Return X_train, X_test, y_train, y_test."""
    X = orders[FEATURES]
    y = orders[TARGET]
    return train_test_split(X, y, test_size=test_size,
                            random_state=SEED)
'''

(PKG / "data.py").write_text(data_py, encoding="utf-8")
print("wrote", PKG / "data.py", f"({len(data_py)} characters)")
```

```
wrote work/delivery/data.py (604 characters)
```

Notice what moved and what did not. The code is the same code from P02. Only its address cha

Step 4 --- Write features.py

One job: judging and describing a single order. No file reading, no model training.

In [6]:
```
features_py = '''"""Checks and descriptions for one delivery order."""

TRAFFIC_LEVELS = (1, 2, 3)


def minutes_per_km(delivery_min, distance_km):
    """How many minutes each kilometre took."""
    if distance_km <= 0:
        raise ValueError("distance_km must be positive")
    return delivery_min / distance_km
```

```
def describe_order(order):
    """A short sentence a human can read."""
    weather = "in the rain" if order["rain"] else "in dry weather"
    return (f"{order['distance_km']} km, "
            f"{order['prep_time_min']} min prep, "
            f"traffic {order['traffic_level']}, {weather}")
'''

(PKG / "features.py").write_text(features_py, encoding="utf-8")
print("wrote", PKG / "features.py")
```

```
wrote work/delivery/features.py
```

Step 5 --- Write model.py

One job: everything about the model itself --- training it, scoring it, saving it to disk and loading it

joblib is the standard way to save a trained scikit-learn model to a file. Saving matters: training a every time you need a prediction would be absurd.

In [7]:
```
model_py = '''"""Training, scoring, saving and loading the model."""

import joblib
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error


def train_model(X_train, y_train):
    """Fit a linear regression and hand it back."""
    return LinearRegression().fit(X_train, y_train)


def evaluate(model, X_test, y_test):
    """Mean absolute error, in minutes."""
    return mean_absolute_error(y_test, model.predict(X_test))


def save_model(model, path):
    """Write a trained model to disk."""
    joblib.dump(model, path)
    return path


def load_model(path):
    """Read a trained model back from disk."""
    return joblib.load(path)
'''

(PKG / "model.py").write_text(model_py, encoding="utf-8")
print("wrote", PKG / "model.py")
```

```
wrote work/delivery/model.py
```

Step 6 --- Add **init**.py to make it a package

A folder of `.py` files is just a folder. Add a file called `__init__.py` and Python treats it as a p you can import.

It can be empty. Ours will list what the package offers, which is good manners.

In [8]:
```
init_py = '''"""Delivery-time prediction for SCSE3040."""

__version__ = "0.1.0"

from .data import FEATURES, TARGET, load_orders, split_orders
from .model import evaluate, load_model, save_model, train_model

__all__ = [
    "FEATURES", "TARGET", "load_orders", "split_orders",
    "train_model", "evaluate", "save_model", "load_model",
]
'''

(PKG / "__init__.py").write_text(init_py, encoding="utf-8")

for f in sorted(PKG.glob("*.py")):
    print(f"  {f.name:<15} {f.stat().st_size:>5} bytes")
```

```
 __init__.py       327 bytes
 data.py           604 bytes
 features.py       594 bytes
 model.py          670 bytes
 validate.py       349 bytes
```

Step 7 --- Import your own package and use it

Python looks for packages in a list of folders called `sys.path`. Our package is inside `work/` add that folder to the list.

The `fresh_import` helper below matters: Python caches modules, so after you *edit* a file you tell it to look again. This trips up everybody who writes modules from a notebook.

In [9]:
```
import importlib
import shutil
import sys

if str(WORK.resolve()) not in sys.path:
    sys.path.insert(0, str(WORK.resolve()))


def fresh_import(name):
    """Import a module, reloading it if it changed on disk.

    The first line deletes Python's compiled cache. Python decides
    whether that cache is stale from the file's size and its
    timestamp to the nearest second -- so an edit that keeps the
    length the same can be missed entirely. Deleting it is simplest.
    """
    shutil.rmtree(PKG / "__pycache__", ignore_errors=True)
    importlib.invalidate_caches()
    module = importlib.import_module(name)
    return importlib.reload(module)


delivery_data = fresh_import("delivery.data")
delivery_model = fresh_import("delivery.model")
```

```
orders = delivery_data.load_orders(DATA)
X_train, X_test, y_train, y_test = delivery_data.split_orders(orders)
model = delivery_model.train_model(X_train, y_train)
mae = delivery_model.evaluate(model, X_test, y_test)

print(f"rows {len(orders)}  train {len(X_train)}  test {len(X_test)}")
print(f"MAE {mae:.2f} minutes")
```

```
rows 600  train 480  test 120
MAE 1.92 minutes
```

Four lines of your own library did what a screen of notebook cells did in P02 --- and this time son else can run it.

Step 8 --- Save the trained model to a file

Training takes time. Predicting should not repeat it. Save the trained model once; load it whenev need a prediction.

In [10]:
```
model_path = WORK / "model.joblib"
delivery_model.save_model(model, model_path)

reloaded = delivery_model.load_model(model_path)
same = float(reloaded.predict(X_test.head(1))[0])
original = float(model.predict(X_test.head(1))[0])

print(f"saved to {model_path} "
      f"({model_path.stat().st_size:,} bytes)")
print(f"original model predicts {original:.2f}")
print(f"reloaded model predicts {same:.2f}")
print("identical:", abs(same - original) < 1e-9)
```

```
saved to work/model.joblib (967 bytes)
original model predicts 47.65
reloaded model predicts 47.65
identical: True
```

That file is the thing P07 will serve and P08 will put in a container. From here on, "the model" me file.

Step 9 --- Write the script a human runs

Finally, the entry point: an ordinary Python file that a person, or a scheduled job, or a container, with one command.

The `if __name__ == "__main__":` line means "only do this when somebody runs this file not when it is imported".

In [11]:
```
train_py = '''"""Train the delivery-time model and report its error."""

import sys
from pathlib import Path

from delivery import (evaluate, load_orders, save_model,
                      split_orders, train_model)
```

```
HERE = Path(__file__).resolve().parent
DEFAULT_DATA = HERE.parent.parent / "data" / "delivery_times.csv"


def main():
    data_path = Path(sys.argv[1]) if len(sys.argv) > 1 else DEFAULT_DATA

    orders = load_orders(data_path)
    X_train, X_test, y_train, y_test = split_orders(orders)
    model = train_model(X_train, y_train)
    mae = evaluate(model, X_test, y_test)

    save_model(model, Path(__file__).parent / "model.joblib")

    print(f"rows: {len(orders)}")
    print(f"MAE: {mae:.2f}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
'''

(WORK / "train.py").write_text(train_py, encoding="utf-8")
print("wrote", WORK / "train.py")
```

```
wrote work/train.py
```

Step 10 --- Run it as a real program

No notebook involved. This is the command you would put in a Dockerfile in P08, or in a pipeline

In [12]:
```
import subprocess

def run_script(name, args=(), cwd=WORK):
    """Run a python file the way a terminal would, and show output."""
    done = subprocess.run(
        [sys.executable, name, *args],
        cwd=cwd, capture_output=True, text=True,
    )
    print(f"$ python {name} {' '.join(args)}".rstrip())
    print(done.stdout.strip() or "(nothing printed)")
    if done.returncode != 0:
        print("STDERR:", done.stderr.strip()[-800:])
    print("exit code:", done.returncode)
    return done

result = run_script("train.py", [str(DATA.resolve())])
```

```
$ python train.py /Users/archit/Desktop/mlops-lab/SCSE3040-Lab/data/
delivery_times.csv
rows: 600
MAE: 1.92
exit code: 0
```

Exit code `0` means success. Any other number means failure --- and that is exactly how a pipel

P11 will decide whether to carry on or stop.

Your turn

The walkthrough above is finished. Now you write some code.

There are **3 tasks**. Each one is small. Each one has a hint. Do them in order.

Where you see `# TODO`, replace that line with your own code. Do not delete the variable name left of the `=` sign --- the self-check at the bottom looks for exactly that name.

When you have tried all three, run the **self-check** cell at the end. It prints a table telling you whic are correct. You can run it as many times as you like.

Task T1 --- Add a function to the package

Add a function `average_speed_kmph(distance_km, delivery_min)` to `delivery/features.py`. It returns the average speed in kilometres per hour.

Speed in km/h is `distance_km / (delivery_min / 60)`.

You must **append** it to the existing file, not replace the file, then re-import the module and store t of `average_speed_kmph(10, 30)` in `T1_speed`.

> **Hint.** Build the new function as a string and use `open(PKG / 'features.py', 'a', encoding='utf-8')` -- the `'a'` means append. Then `feat = fresh_import('delivery.features')` and call `feat.average_speed_kmph(10, 30)`. Ten km in thirty minutes is 20 km/h.

In [13]:
```
new_function = '''

def average_speed_kmph(distance_km, delivery_min):
    "Average speed of a delivery, in kilometres per hour."
    return distance_km / (delivery_min / 60)
'''

with open(PKG / "features.py", "a", encoding="utf-8") as fh:
    fh.write(new_function)

feat = fresh_import("delivery.features")

T1_speed = feat.average_speed_kmph(10, 30)

print("10 km in 30 minutes =", T1_speed, "km/h")
```

```
10 km in 30 minutes = 20.0 km/h
```

Task T2 --- Reject an impossible order

Write a **new module** `delivery/validate.py` containing one function, `is_valid_order(order)`, which takes a dictionary and returns `True` only if all four of thes

- `distance_km` is greater than 0
- `prep_time_min` is 0 or more
- `traffic_level` is 1, 2 or 3
- `rain` is 0 or 1

Anything else returns `False`. Import it and store the module in `T2_validate`.

> **Hint.** Build the module as one big string and `write_text` it, exactly the way Steps 3-5 did. Inside the function, use `order["..."]` to read each value, and `return False` as soon as any rule is broken. Then `T2_validate = fresh_import('delivery.validate')`.

In [14]:
```
validate_py = '''"Is this order usable?"


def is_valid_order(order):
    "True if the order passes every rule."
    if order["distance_km"] <= 0:
        return False
    if order["prep_time_min"] < 0:
        return False
    if order["traffic_level"] not in (1, 2, 3):
        return False
    if order["rain"] not in (0, 1):
        return False
    return True
'''

(PKG / "validate.py").write_text(validate_py, encoding="utf-8")

T2_validate = fresh_import("delivery.validate")

good = {"distance_km": 5.0, "prep_time_min": 20,
        "traffic_level": 2, "rain": 0}

if T2_validate is None:
    print("Not done yet -- fill in the two TODOs above.")
else:
    print("good order -> ", T2_validate.is_valid_order(good))
```

```
good order ->  True
```

Task T3 --- A second command-line script

Write `work/predict.py`: a script that loads the saved model from `work/model.joblib` a one prediction for a 7 km order with 25 minutes of preparation, traffic level 3, no rain.

It must print a line that starts with `PREDICTION:` followed by the number of minutes.

Run it with `run_script("predict.py")` and store the finished process in `T3_run`.

> **Hint.** Copy the shape of `train.py` from Step 9. Import `load_model` from `delivery`, build a one-row `pandas` DataFrame with the four columns, call `.predict(...)`, and print `f"PREDICTION: {minutes:.1f}"`. Do not forget the `if __name__ == "__main__":` guard.

In [15]:
```
predict_py = '''"Predict one delivery, from the command line."

from pathlib import Path

import pandas as pd

from delivery import load_model


def main():
    model = load_model(Path(__file__).parent / "model.joblib")
    one = pd.DataFrame([{
        "distance_km": 7.0,
        "prep_time_min": 25,
        "traffic_level": 3,
        "rain": 0,
    }])
    minutes = float(model.predict(one)[0])
    print(f"PREDICTION: {minutes:.1f}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
'''

(WORK / "predict.py").write_text(predict_py, encoding="utf-8")

T3_run = run_script("predict.py")
```

```
$ python predict.py
PREDICTION: 56.5
exit code: 0
```

Self-check

Run the cell below to mark your work.

In [16]:
```
# ----------------------------------------------------------------------
# SELF-CHECK --- run this when you have attempted the tasks above.
# It never breaks your notebook. A task you have not done yet simply
# shows FAIL.
# ----------------------------------------------------------------------

_results = []


def _check(label, fn):
```

```
    """Evaluate one graded condition without ever raising."""
    try:
        ok = bool(fn())
    except Exception:
        ok = False
    _results.append((label, ok))


_check('T1 | delivery.features now offers average_speed_kmph', lambda: ha
_check('T1 | 10 km in 30 minutes is 20 km/h', lambda: abs(float(T1_speed)
_check('T1 | it still works for another order', lambda: abs(float(feat.av
_check('T2 | delivery/validate.py exists and imports', lambda: (PKG / 'va
_check('T2 | a sensible order is accepted', lambda: T2_validate.is_valid_
_check('T2 | a zero distance is rejected', lambda: T2_validate.is_valid_o
_check('T2 | traffic level 9 and rain 5 are both rejected', lambda: T2_va
_check('T3 | predict.py ran and exited successfully', lambda: T3_run.retu
_check('T3 | it printed a PREDICTION line with a sensible number', lambda

print("======================================================================")
print("SELF-CHECK   Practical 04 --- From Notebook to Package")
print("======================================================================")
for _label, _ok in _results:
    print(f"  [{'PASS' if _ok else 'FAIL'}]  {_label}")
print("----------------------------------------------------------------------")
_passed = sum(1 for _, _ok in _results if _ok)
print(f"  {_passed} of {len(_results)} checks passed")
print("======================================================================")
if _passed == len(_results):
    print("Well done. Save the notebook and submit it.")
else:
    print("Read the FAIL lines above, fix those tasks, run this cell agai
```

```
======================================================================
SELF-CHECK   Practical 04 --- From Notebook to Package
======================================================================
  [PASS]  T1 | delivery.features now offers average_speed_kmph
  [PASS]  T1 | 10 km in 30 minutes is 20 km/h
  [PASS]  T1 | it still works for another order
  [PASS]  T2 | delivery/validate.py exists and imports
  [PASS]  T2 | a sensible order is accepted
  [PASS]  T2 | a zero distance is rejected
  [PASS]  T2 | traffic level 9 and rain 5 are both rejected
  [PASS]  T3 | predict.py ran and exited successfully
  [PASS]  T3 | it printed a PREDICTION line with a sensible number
----------------------------------------------------------------------
  9 of 9 checks passed
======================================================================
Well done. Save the notebook and submit it.
```

- **data.py**: Loads the delivery CSV into a table and splits it into training and testing sets.
- **features.py**: Checks and describes a single delivery order, such as its minutes per km, ave speed and a readable summary.
- **model.py**: Trains the linear regression model, scores it, and saves and loads it from disk.
- **validate.py**: Decides whether a single order is usable by checking that each value is in its a range.
- **init.py**: Marks the `delivery` folder as a package and lists the functions it offers for impor

---

What to submit

1. This notebook, with every cell run and its output visible.
2. The `work/delivery/` folder you built, zipped.
3. In a markdown cell, one sentence per module saying what its single job is.

Name your file `PXX_<your-roll-number>.ipynb` before you upload it.

How this practical is marked

What is marked	Marks
Walkthrough run end to end, package builds and imports	3
Task T1 --- a new function added to the package	2
Task T2 --- validation logic that rejects bad orders	3
Task T3 --- a script that runs from the command line	2
Total	**10**

Read more

- Python docs --- Modules and packages --- https://docs.python.org/3/tutorial/modules.html
- Python docs --- The module search path --- https://docs.python.org/3/tutorial/modules.html#module-search-path
- joblib --- Persistence --- https://joblib.readthedocs.io/en/stable/persistence.html
- scikit-learn --- Model persistence --- https://scikit-learn.org/stable/model_persistence.html

SCSE3040 Machine Learning Operations

Practical 05 --- Configuration and Automated Testing

1. Student details

Name	Archit
Roll number	S24CSEU1049
Batch	12
Date of lab session	_____

2. Repository

https://github.com/testing-archit/SCSE3040-Lab

3. config.yaml

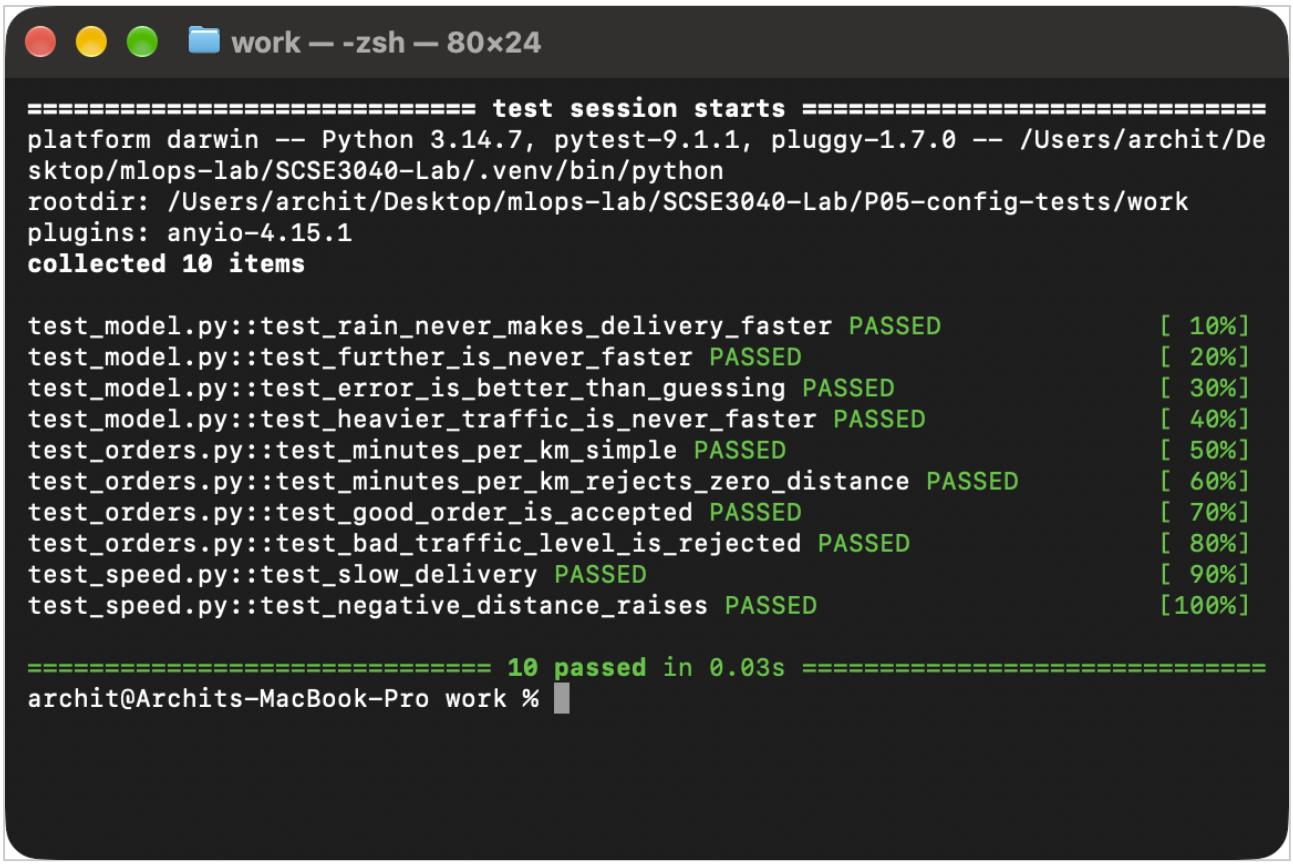
```
# Settings for the delivery-time model.
# Change values here, never in the code.

data:
  path: ../data/delivery_times.csv
  features:
    - distance_km
    - prep_time_min
    - traffic_level
    - rain
  target: delivery_min

split:
  test_size: 0.2
  seed: 42

model:
  name: linear_regression
```

4. pytest output



5. Result before and after the refactor

Before the refactor (settings hard-coded): MAE 1.92 minutes; now (settings read from config.yaml): MAE 1.92 minutes.

SCSE3040 Machine Learning Operations

Practical 06 --- Remembering Every Experiment (MLflow)

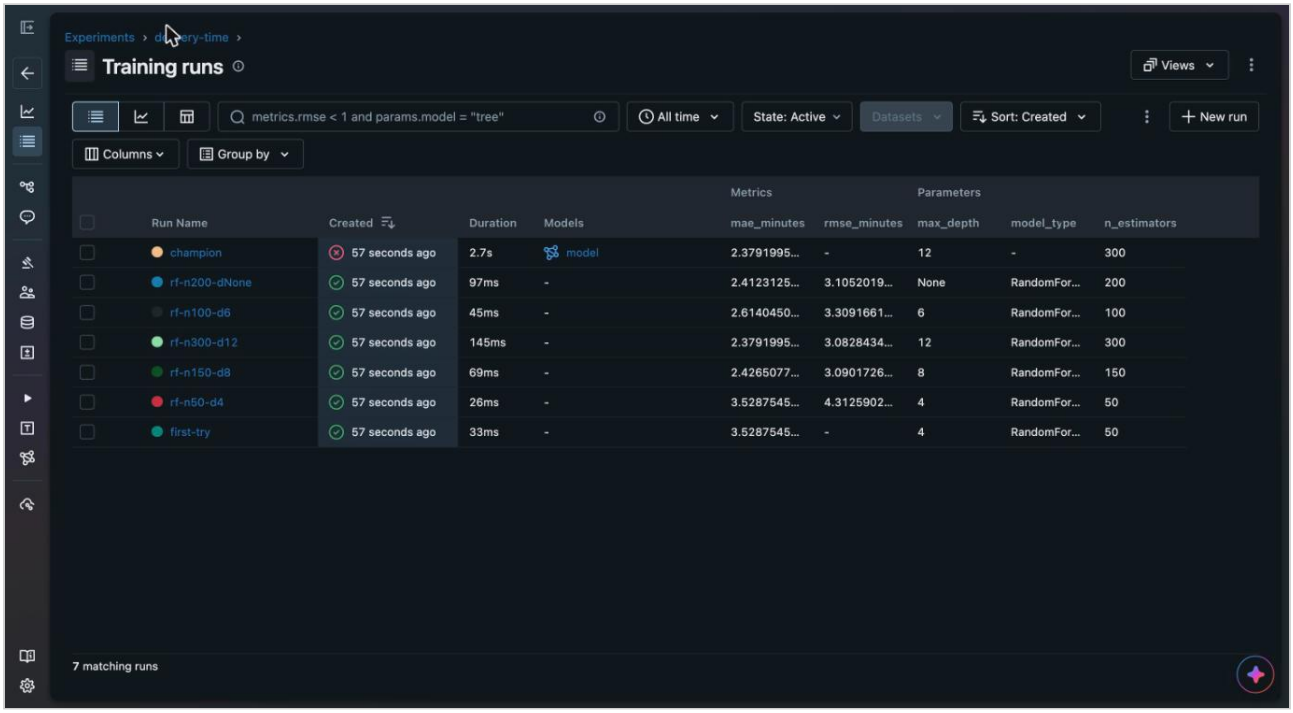
1. Student details

Name	Archit
Roll number	S24CSEU1049
Batch	12
Date of lab session	_____

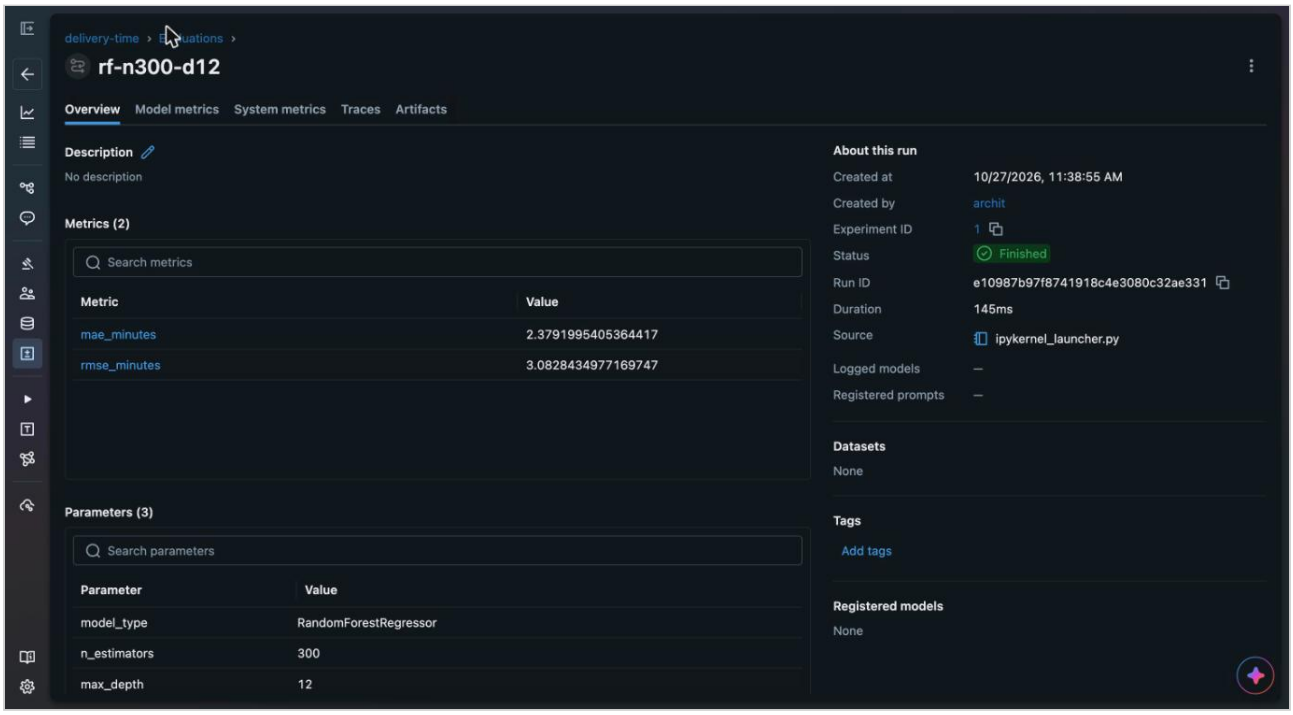
2. Repository

https://github.com/testing-archit/SCSE3040-Lab

3. MLflow runs table --- experiment `delivery-time`



4. Run detail page --- selected best run



5. Selected run

Run ID `e10987b97f8741918c4e3080c32ae331` (rf-n300-d12)

Metric mae_minutes = 2.379 (rmse_minutes = 3.083)

I chose rf-n300-d12 because it has the lowest test MAE of all the sweep runs (2.379 minutes), beating the next best, rf-n200-dNone at 2.412, and the worst, rf-n50-d4 at 3.529.

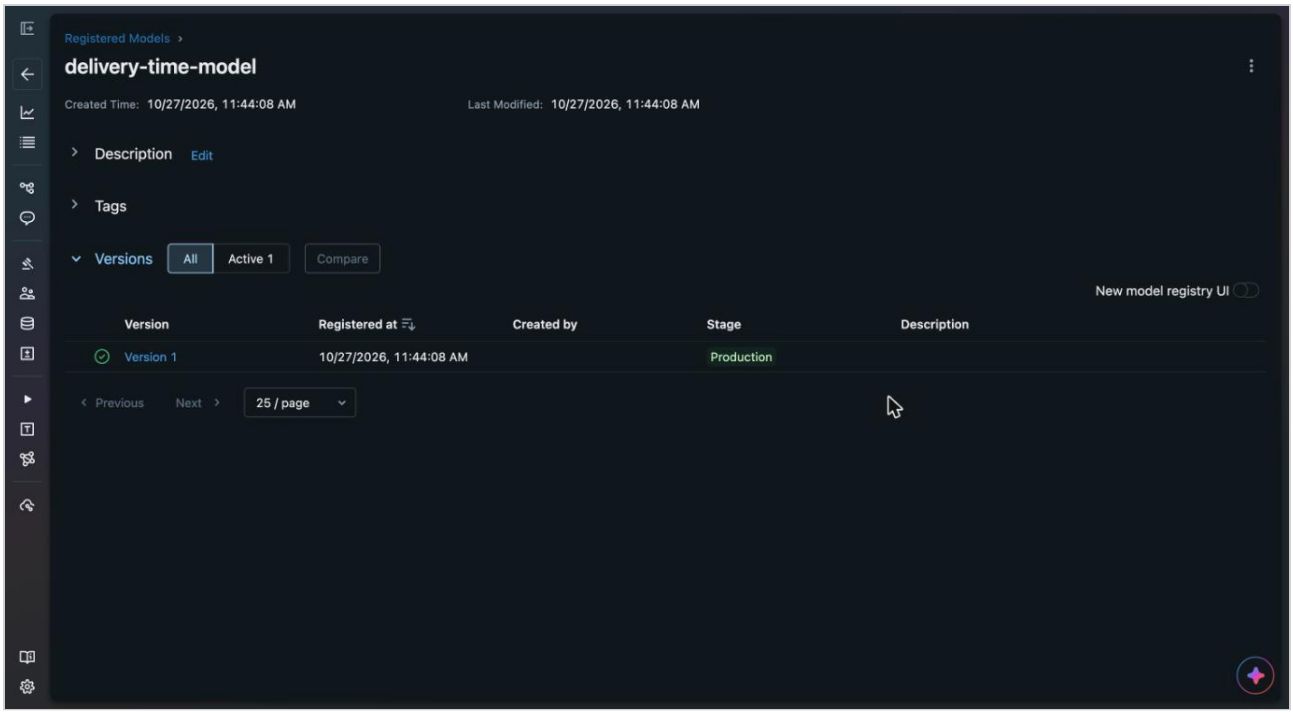
SCSE3040 Machine Learning Operations

Practical 07 --- Serving a Registered Model

1. Student details

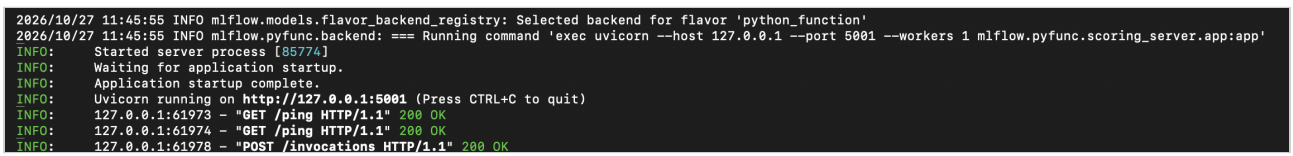
Name	Archit
Roll number	S24CSEU1049
Batch	12
Date of lab session	_____

2. MLflow Model Registry --- delivery-time-model



3. Terminal running mlflow models serve

```
mlflow models serve -m models:/delivery-time-model/Production -p 5001 --env-manager local
```



4. Test request with curl and the prediction returned

```
architt@Architts-MacBook-Pro ~ % curl -s -X POST http://127.0.0.1:5001/invocations \
>   -H 'Content-Type: application/json' \
>   -d '{"dataframe_split": {"columns": ["distance_km", "prep_time_min", "traffic_level", "rain"], "data": [[5.0, 20, 2, 0]]}}'; echo
{"predictions": [42.8869888888888844]}
architt@Architts-MacBook-Pro ~ %
```

5. Served model

Registered model name	`delivery-time-model`
Version promoted	1 (stage: Production)
Source run ID	`e10987b97f8741918c4e3080c32ae331` (rf-n300-d12)
Port served on	5001 (http://127.0.0.1:5001/invocations)

SCSE3040 Machine Learning Operations

Practical 08 --- Deploying the Prediction Service to the Cloud

1. Student details

Name	Archit
Roll number	S24CSEU1049
Batch	12
Date of lab session	_____

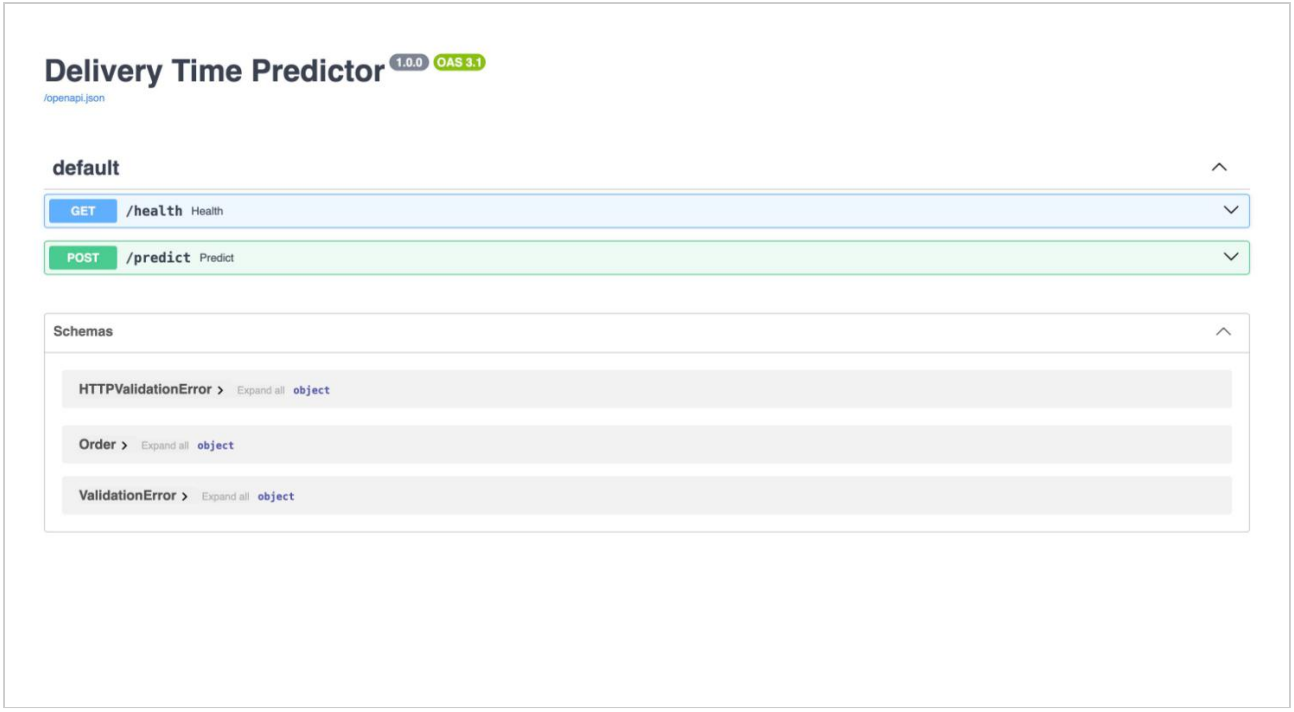
2. Repository

https://github.com/testing-archit/SCSE3040-Lab

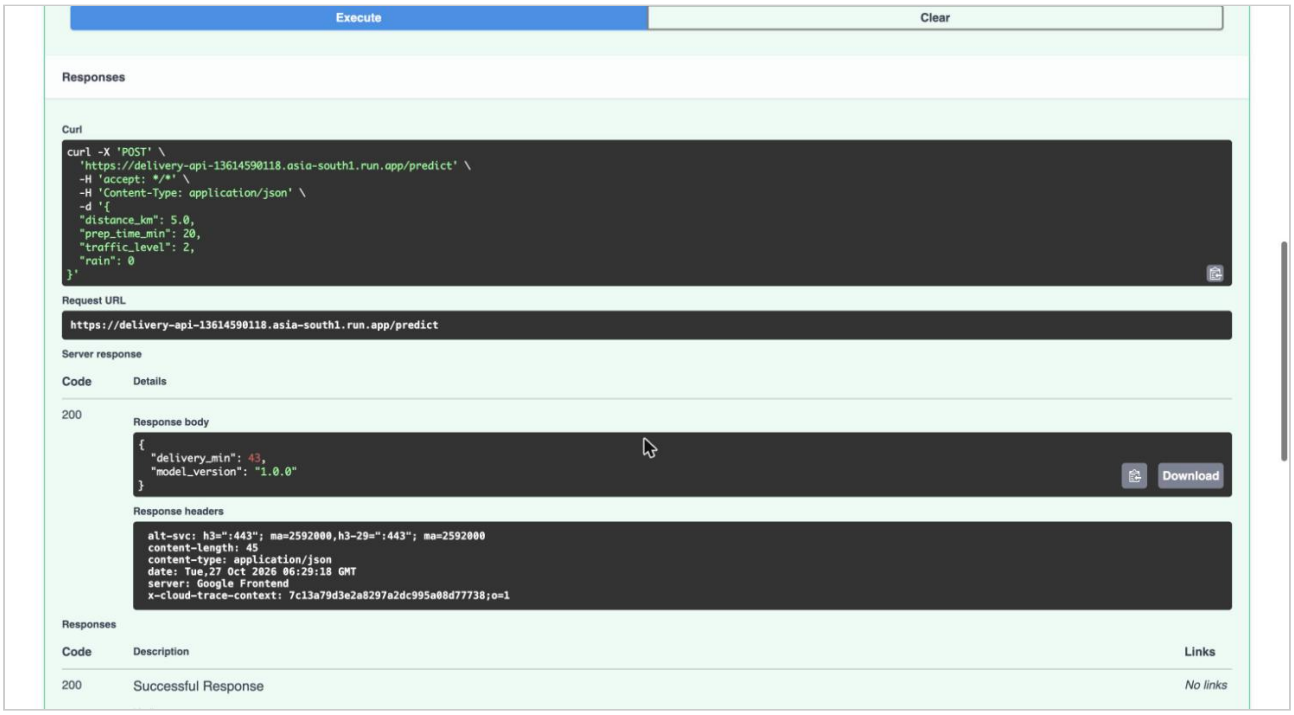
3. Public URL of the deployed service

https://delivery-api-13614590118.asia-south1.run.app
Interactive docs: https://delivery-api-13614590118.asia-south1.run.app/docs

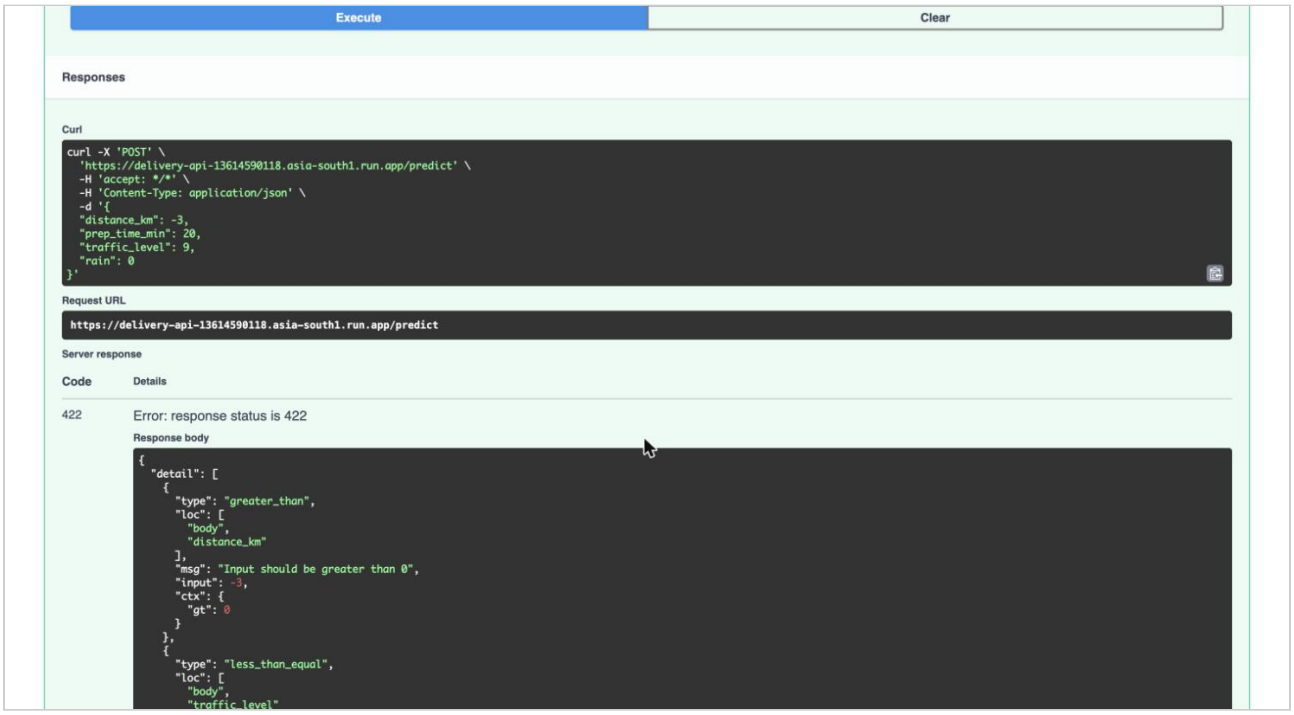
4. Interactive documentation (/docs) showing the /predict endpoint



5. Successful call to /predict on the public URL



6. Rejected call --- request validation refused bad input (HTTP 422)



7. Cloud platform

Google Cloud Run (Google Cloud Platform), region asia-south1 (Mumbai).

Practical 09 --- Describing the Service to a Clu

Deployment, Service and probes, written as YAML and checked before they run

SCSE3040 Machine Learning Operations · Bennett University · Session 2026-27

Lectures this follows	L15-L17
Course Outcome	CO4
Lab duration	120 minutes
Memory needed	about 200 MB (fine on a 4 GB or 8 GB machine)
Extra software	nothing beyond the course venv
Marks	10

Aim

1. Explain what a Deployment and a Service each do, in plain words.
2. Write both as YAML manifests for the delivery-time container.
3. Check a manifest is correct before it ever reaches a cluster.
4. Add readiness and liveness probes, and say what each one is for.

What you need to know first

In P08 you ran one container with `docker run`. That works until you need a second copy, or t
machine reboots, or the container dies at two in the morning. Somebody has to notice and start i

Kubernetes is the software that does the noticing. You do not tell it what to do; you tell it **what y**
and it works continuously to make reality match. That style is called **declarative**, and it is the sin
behind everything today.

Three objects matter for us.

A **Pod** is one running copy of your container. It is the smallest thing Kubernetes handles. Pods a
disposable: they get killed and replaced, and they get a new address every time.

A **Deployment** is a standing order: *keep three copies of this image running at all times*. If one di
Deployment makes another. If you change the image, it replaces them one at a time so the servi
goes down.

A **Service** is one permanent address in front of those changing Pods. Since Pods come and go
addresses, nothing could reliably call them. The Service is the fixed name that spreads requests
whichever Pods exist now.

The glue between Deployment and Service is **labels**. A Deployment stamps a label on its Pods; Service says "send traffic to anything with this label". If those two strings do not match, everythin appears healthy and nothing works. That is the most common Kubernetes mistake in the world, will fix it in Task T3.

Finally, **probes**. Kubernetes cannot guess what "working" means for your service, so you tell it. **readiness** probe asks *should traffic come yet?* --- a model that takes 30 seconds to load should receive requests at second 2. A **liveness** probe asks *are you still alive?* --- if it fails, the containe restarted. Confusing the two is how one slow pod becomes an endless restart loop.

```
---

## Before you start

- Practical P08 is finished. You have an image called `delivery-
api:1.0`.
```

- You have seen YAML in P05. Remember: spaces only, never tabs.
- `kubectl` and a cluster are **optional** today. Every graded task is about writing correct mar which you can do on any machine. If Docker Desktop has Kubernetes enabled, the noteboo also run the real commands for you.

How to run a cell

Click on a grey code cell, then press **Shift + Enter**. The cell runs and the cursor moves to th one. A number appears in the `[ ]` on the left when the cell has finished.

Run the cells in order, from the top. A later cell almost always uses something an earlier created. If you jump ahead you will see a `NameError`, which just means "you have not ma thing yet".

If everything goes wrong, use the menu: **Kernel -> Restart Kernel and Clear All Outputs**, start again from the first cell. Nothing is damaged by doing this.

In [ ]:
```
# Step 0 --- check the workbench before we start.
# This cell only looks; it changes nothing. Run it and read the last line

import sys
from pathlib import Path

print("Python  :", sys.version.split()[0])
print("Folder  :", Path.cwd().name)

_missing = []
for _name in ['yaml']:
    try:
        __import__(_name)
    except ImportError:
        _missing.append(_name)
```

```
for _name in ['yaml']:
    _mark = "missing" if _name in _missing else "ok"
    print(f"  {_name:<14} {_mark}")

if _missing:
    print()
    print("STOP. Some libraries are missing:", ", ".join(_missing))
    print("Ask your instructor to run the setup in labs/SETUP.md.")
else:
    print()
    print("All good. You can carry on to Step 1.")
```

Walkthrough

Read each step, then run its cell.

Step 1 --- Check whether a cluster is available

This is optional today. If `kubectl` is installed the notebook will run real commands for you; if n
everything else still works, because writing correct manifests is the actual skill.

In [ ]:
```
import shutil
import subprocess
from pathlib import Path

import yaml

WORK = Path("work")
WORK.mkdir(exist_ok=True)

KUBECTL_OK = shutil.which("kubectl") is not None
CLUSTER_OK = False

if KUBECTL_OK:
    probe = subprocess.run(
        ["kubectl", "cluster-info"], capture_output=True,
        text=True, timeout=30,
    )
    CLUSTER_OK = probe.returncode == 0
    print("kubectl found. Cluster reachable:", CLUSTER_OK)
else:
    print("kubectl is not installed on this machine.")
    print("That is fine. Every task today is about writing correct")
    print("YAML, and the notebook checks your YAML itself.")

print()
print("KUBECTL_OK =", KUBECTL_OK, "  CLUSTER_OK =", CLUSTER_OK)


def kubectl(*args):
    """Run one kubectl command, or explain why it was skipped."""
    if not CLUSTER_OK:
        print(f"SKIPPED (no cluster): kubectl {' '.join(args)}")
        return None
```

```
    print(f"$ kubectl {' '.join(args)}")
    done = subprocess.run(["kubectl", *args], capture_output=True,
                          text=True, timeout=120)
    print((done.stdout + done.stderr).strip()[-2000:])
    print("-" * 60)
    return done
```

Step 2 --- Write the Deployment

Read it top to bottom. Every manifest has the same four opening keys: `apiVersion`, `kind`, `metadata`, `spec`.

The part that catches everybody: there are **two** `metadata` blocks. The outer one names the Deployment. The inner one, under `template`, names the Pods it will create. The `selector` middle says which Pods this Deployment is responsible for --- and it must match the template's l exactly.

In [ ]:
```
deployment_yaml = """apiVersion: apps/v1
kind: Deployment
metadata:
  name: delivery-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: delivery-api
  template:
    metadata:
      labels:
        app: delivery-api
    spec:
      containers:
        - name: api
          image: delivery-api:1.0
          ports:
            - containerPort: 8000
          resources:
            requests:
              memory: "256Mi"
              cpu: "100m"
            limits:
              memory: "512Mi"
              cpu: "500m"
"""

(WORK / "deployment.yaml").write_text(deployment_yaml,
                                      encoding="utf-8")
print(deployment_yaml)
```

`replicas: 3` is the standing order. `resources` tells the cluster how much this container ne it can decide which machine has room --- and stops one runaway pod from eating a whole node.

Step 3 --- Check the YAML before anybody runs it

A manifest is just text. The fastest way to catch a mistake is to load it in Python and look at it --- r
cluster needed.

This habit is worth more than it looks: in P11 a pipeline will do exactly this check automatically, b
deploying.

In [ ]:
```
with open(WORK / "deployment.yaml", encoding="utf-8") as fh:
    deployment = yaml.safe_load(fh)

print("kind      :", deployment["kind"])
print("name      :", deployment["metadata"]["name"])
print("replicas  :", deployment["spec"]["replicas"])

selector = deployment["spec"]["selector"]["matchLabels"]
pod_labels = deployment["spec"]["template"]["metadata"]["labels"]

print("selector  :", selector)
print("pod labels:", pod_labels)
print()
print("labels match:", selector == pod_labels,
      "  <- if this is False, nothing will work")
```

Step 4 --- Write the Service

The Service is much shorter. Two things matter.

`selector` must match the Pod labels from the Deployment --- that is how it finds them.

`port` is the address the Service answers on; `targetPort` is the port inside the container. Tl
allowed to differ, and confusing them is the second most common mistake.

In [ ]:
```
service_yaml = """apiVersion: v1
kind: Service
metadata:
  name: delivery-api-svc
spec:
  type: ClusterIP
  selector:
    app: delivery-api
  ports:
    - port: 80
      targetPort: 8000
"""

(WORK / "service.yaml").write_text(service_yaml, encoding="utf-8")
print(service_yaml)
```

`ClusterIP` means "reachable inside the cluster only". That is the right default. `NodePort` a
`LoadBalancer` open it up further, and you should only reach for them when you actually neec

Step 5 --- Prove the Service can find the Pods

Load both files and compare the two strings that must agree. This three-line check would have s great many careers' worth of afternoons.

In [ ]:
```
with open(WORK / "service.yaml", encoding="utf-8") as fh:
    service = yaml.safe_load(fh)

svc_selector = service["spec"]["selector"]

print("Service selects :", svc_selector)
print("Pods are labelled:", pod_labels)
print()
if svc_selector == pod_labels:
    print("MATCH. Traffic will reach the pods.")
else:
    print("MISMATCH. The Service would find nothing,")
    print("and Kubernetes would not warn you.")
```

Step 6 --- Add the health probes

Now teach the cluster what healthy means. Both probes call the `/health` endpoint you wrote

Look at the two `initialDelaySeconds` values and think about why they differ. Readiness st asking early, because we want traffic as soon as possible. Liveness waits much longer, because container that is merely still starting up is a disaster.

In [ ]:
```
probes = {
    "readinessProbe": {
        "httpGet": {"path": "/health", "port": 8000},
        "initialDelaySeconds": 5,
        "periodSeconds": 10,
    },
    "livenessProbe": {
        "httpGet": {"path": "/health", "port": 8000},
        "initialDelaySeconds": 30,
        "periodSeconds": 20,
        "failureThreshold": 3,
    },
}

container = deployment["spec"]["template"]["spec"]["containers"][0]
container.update(probes)

(WORK / "deployment.yaml").write_text(
    yaml.safe_dump(deployment, sort_keys=False), encoding="utf-8"
)

print(yaml.safe_dump(container, sort_keys=False))
```

We edited the manifest as a Python dictionary and wrote it back. That is a genuinely useful trick, how a pipeline changes an image tag automatically in P11.

Step 7 --- What each probe does when it fails

Say this out loud until it sticks.

In [ ]:
```
print("readiness fails -> the pod is taken OUT of the Service")
print("                   no traffic arrives; the pod keeps running")
print()
print("liveness  fails -> the container is KILLED and restarted")
print()
print("So:")
print("  slow to start        -> raise readiness initialDelaySeconds")
print("  hung, needs a kick   -> that is what liveness is for")
print("  only a liveness probe-> traffic hits a pod that is not ready")
```

Step 8 --- Send it to a cluster, if you have one

These are the real commands. They run here only if a cluster is reachable; otherwise you will se printed, which is what you need for your notes.

`apply` is declarative: run it twice and nothing bad happens. It makes reality match the file.

In [ ]:
```
kubectl("apply", "-f", str(WORK / "deployment.yaml"))
kubectl("apply", "-f", str(WORK / "service.yaml"))
kubectl("get", "deployment", "delivery-api")
kubectl("get", "pods", "-l", "app=delivery-api")
kubectl("get", "service", "delivery-api-svc")

print()
print("The commands above, for your notes:")
for line in [
    "kubectl apply -f deployment.yaml",
    "kubectl apply -f service.yaml",
    "kubectl get pods -l app=delivery-api",
    "kubectl describe pod <pod-name>      # read Events at the end",
    "kubectl logs <pod-name>",
    "kubectl rollout status deployment/delivery-api",
    "kubectl rollout undo deployment/delivery-api",
    "kubectl delete -f deployment.yaml -f service.yaml",
]:
    print("   ", line)
```

Step 9 --- Reading a broken deployment

You will meet these four states constantly. Learn them now; it saves hours later.

In [ ]:
```
states = [
    ("Pending", "no machine has room, or a volume is missing"),
    ("ContainerCreating", "pulling the image; normal for a minute"),
    ("ImagePullBackOff", "wrong image name or tag, or no access"),
    ("CrashLoopBackOff", "it starts, dies, restarts, forever"),
    ("Running", "the process started -- which is not the same as working"
]

for state, meaning in states:
    print(f"  {state:<20} {meaning}")
```

```
print()
print("First command every time: kubectl describe pod <name>")
print("Then read the Events list at the very bottom. The reason")
print("is almost always there.")
```

Your turn

The walkthrough above is finished. Now you write some code.

There are **3 tasks**. Each one is small. Each one has a hint. Do them in order.

Where you see `# TODO`, replace that line with your own code. Do not delete the variable name left of the `=` sign --- the self-check at the bottom looks for exactly that name.

When you have tried all three, run the **self-check** cell at the end. It prints a table telling you whic are correct. You can run it as many times as you like.

Task T1 --- Write a Service for a second version

```
Write `work/service-v2.yaml`: a Service named `delivery-api-v2-svc`
that:

- is of type `NodePort`
- selects Pods labelled `app: delivery-api-v2`
- answers on port `80` and forwards to `targetPort` `8000`

Load it back with `yaml.safe_load` and store the dictionary in
`T1_service`.
```

> **Hint.** Copy the Service from Step 4 and change four things: the name, the type, the selector value and nothing else. Write it with `write_text`, then read it back with `yaml.safe_load` so the check can see it.

In [ ]:
```
service_v2 = """apiVersion: v1
kind: Service
metadata:
  name: TODO
spec:
  type: TODO
  selector:
    app: TODO
  ports:
    - port: 0
      targetPort: 0
"""

(WORK / "service-v2.yaml").write_text(service_v2, encoding="utf-8")

# TODO: load the file back into a dictionary
T1_service = None
```

```
print(T1_service)
```

Task T2 --- Probes for a slow model

A new model takes **40 seconds** to load. The Deployment in Step 6 a
liveness probe with `initialDelaySeconds: 30`, which would kill it
mid-load, forever.

Build a dictionary `T2_probes` with two keys, `readinessProbe` and
`livenessProbe`, both calling `/health` on port 8000, where:

- readiness starts after **10** seconds
- liveness starts after **60** seconds --- comfortably past the 40
  needs

> **Hint.** Copy the `probes` dictionary from Step 6 and change the two `initialDelaySeconds` values. Keep the `httpGet` blocks exactly as they are.

In [ ]:
```
# TODO: two probes, both httpGet /health on port 8000,
#       readiness after 10 seconds, liveness after 60.
T2_probes = None

print(T2_probes)
```

Task T3 --- Find the mismatch

The two manifests below look fine and would deploy without a single
warning --- and no traffic would ever reach a pod.

Load them both, find the fault, and fix it by editing the **Service**
so that it selects the Pods the Deployment actually creates.

Put the corrected Service dictionary in `T3_fixed_service`.

> **Hint.** Print `broken_deploy['spec']['template']['metadata']['labels']` and `broken_svc['spec']['selector']` and read them side by side. One word differs. Change the Service's selector to match the Pod labels exactly.

In [ ]:
```
broken_deploy = yaml.safe_load("""
apiVersion: apps/v1
kind: Deployment
metadata:
  name: payments-api
spec:
  replicas: 2
  selector:
    matchLabels:
      app: payments-api
  template:
    metadata:
      labels:
```

```
        app: payments-api
    spec:
      containers:
        - name: api
          image: payments-api:3.1
""")

broken_svc = yaml.safe_load("""
apiVersion: v1
kind: Service
metadata:
  name: payments-svc
spec:
  selector:
    app: payment-api
  ports:
    - port: 80
      targetPort: 8080
""")

# TODO: fix the Service so its selector matches the pod labels
T3_fixed_service = None

print(T3_fixed_service)
```

Self-check

Run the cell below to mark your work.

In [ ]:

```
# ----------------------------------------------------------------------
# SELF-CHECK --- run this when you have attempted the tasks above.
# It never breaks your notebook. A task you have not done yet simply
# shows FAIL.
# ----------------------------------------------------------------------

_results = []


def _check(label, fn):
    """Evaluate one graded condition without ever raising."""
    try:
        ok = bool(fn())
    except Exception:
        ok = False
    _results.append((label, ok))


_check('T1 | it is a Service with the right name', lambda: T1_service['ki
_check('T1 | its type is NodePort and it selects delivery-api-v2', lambda
_check('T1 | port 80 forwards to targetPort 8000', lambda: T1_service['sp
_check('T2 | both probes are present', lambda: set(T2_probes) == {'readin
_check('T2 | both call /health on port 8000', lambda: all(p['httpGet'] ==
_check('T2 | readiness starts at 10s and liveness at 60s', lambda: T2_pro
_check('T2 | liveness waits longer than readiness', lambda: T2_probes['li
_check('T3 | the Service selector now matches the pod labels', lambda: T3
_check('T3 | the selector is app: payments-api, spelled correctly', lambd
```

```
_check('T3 | nothing else about the Service was damaged', lambda: T3_fixe

print("======================================================================
print("SELF-CHECK   Practical 09 --- Describing the Service to a Cluster"
print("======================================================================
for _label, _ok in _results:
    print(f"  [{'PASS' if _ok else 'FAIL'}]  {_label}")
print("----------------------------------------------------------------------
_passed = sum(1 for _, _ok in _results if _ok)
print(f"  {_passed} of {len(_results)} checks passed")
print("======================================================================
if _passed == len(_results):
    print("Well done. Save the notebook and submit it.")
else:
    print("Read the FAIL lines above, fix those tasks, run this cell agai
```

What to submit

1. This notebook, with every cell run and its output visible.
2. Your `work/deployment.yaml` and `work/service.yaml`.
3. In a markdown cell, two sentences: what happens when a readiness probe fails, and what h when a liveness probe fails.

Name your file `PXX_<your-roll-number>.ipynb` before you upload it.

How this practical is marked

What is marked	Marks
Walkthrough run end to end, manifests written and checked	3
Task T1 --- a correct Service manifest	2
Task T2 --- probes added with sensible delays	3
Task T3 --- the label mismatch found and fixed	2
Total	**10**

Read more

- Kubernetes --- Deployments --- https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
- Kubernetes --- Service --- https://kubernetes.io/docs/concepts/services-networking/service/
- Kubernetes --- Configure liveness, readiness and startup probes --- https://kubernetes.io/do configure-pod-container/configure-liveness-readiness-startup-probes/
- Kubernetes --- kubectl quick reference --- https://kubernetes.io/docs/reference/kubectl/quick reference/

Practical 10 --- Logs You Can Actually Search

Levels, structured JSON events, and what must never be written down

SCSE3040 Machine Learning Operations · Bennett University · Session 2026-27

Lectures this follows	L18
Course Outcome	CO5
Lab duration	120 minutes
Memory needed	about 350 MB (fine on a 4 GB or 8 GB machine)
Extra software	nothing beyond the course venv
Marks	10

Aim

1. Explain why `print()` is not good enough once software is deployed.
2. Use logging levels so you can turn detail up and down without editing code.
3. Write one JSON object per event, so logs can be searched by machine.
4. Recognise what must never appear in a log, and mask it.

What you need to know first

In P08 your service ran inside a container. Suppose at nine at night a customer complains that a prediction was wrong. What do you actually have to look at?

The container is gone. Its filesystem went with it. `print()` output vanished with the process. Y nothing.

That is what **logging** is for. A log is a permanent, timestamped record of what your software did. ideas make one useful.

Levels. Not every message matters equally. Python gives you five: `DEBUG` (detail for develope `INFO` (normal events worth recording), `WARNING` (something odd but survivable), `ERROR` (th request failed), and `CRITICAL` (the service is going down). You set a **threshold** and everythin it is discarded. That is how the same code can be quiet in production and chatty while you are de --- with no code change at all.

Structure. A human reads `"prediction took 42ms"`. A machine cannot. With fifty containe producing thousands of lines an hour, nobody reads logs by eye --- a tool searches them. So wri **JSON** object per line, with named fields: `{"event": "prediction", "duration_ms": 4 "distance_km": 5.0}`. Now "show me every prediction slower than 200 ms" is a query, not afternoon.

Where they go. In a container you do **not** write to a log file. Write to standard output, and let the collect it. That is what `kubectl logs` reads, and what a collector like FluentD or Fluent Bit sh central store such as the ELK stack. A log file inside a container dies with the container, which is the problem you were trying to solve.

One rule that is not technical. A log is a permanent record that many people can read. Names, p numbers, exact home addresses, card numbers --- none of it belongs there. You will practise ren today.

Before you start

- Practical P07 is finished. You have a service with a /predict endpoint.
- You know what a dictionary is and what JSON looks like.

How to run a cell

Click on a grey code cell, then press **Shift + Enter**. The cell runs and the cursor moves to th one. A number appears in the `[ ]` on the left when the cell has finished.

Run the cells in order, from the top. A later cell almost always uses something an earlier c created. If you jump ahead you will see a `NameError`, which just means "you have not ma thing yet".

If everything goes wrong, use the menu: **Kernel -> Restart Kernel and Clear All Outputs**, start again from the first cell. Nothing is damaged by doing this.

In [ ]:
```
# Step 0 --- check the workbench before we start.
# This cell only looks; it changes nothing. Run it and read the last line

import sys
from pathlib import Path

print("Python  :", sys.version.split()[0])
print("Folder  :", Path.cwd().name)

_missing = []
for _name in ['numpy', 'pandas', 'sklearn', 'joblib']:
    try:
        __import__(_name)
    except ImportError:
        _missing.append(_name)

for _name in ['numpy', 'pandas', 'sklearn', 'joblib']:
    _mark = "missing" if _name in _missing else "ok"
    print(f"  {_name:<14} {_mark}")

if _missing:
    print()
```

```
    print("STOP. Some libraries are missing:", ", ".join(_missing))
    print("Ask your instructor to run the setup in labs/SETUP.md.")
else:
    print()
    print("All good. You can carry on to Step 1.")
```

Step 0b --- the dataset

Every practical in this course uses the same 600 food deliveries. The next cell makes sure the fil
there.

In [ ]:
```
# The delivery-time dataset every practical in this course uses.
# If the file is missing we build it again from the same seed, so every
# student in the room gets byte-for-byte the same 600 rows.

import csv
from pathlib import Path

import numpy as np

SEED = 42
N_ROWS = 600
DATA = Path("..") / "data" / "delivery_times.csv"


def make_delivery_csv(path=DATA):
    """Write the 600-row delivery dataset. Same formula as the lectures."
    rng = np.random.default_rng(SEED)
    distance_km = np.round(rng.uniform(0.5, 12.0, N_ROWS), 2)
    prep_time_min = np.round(rng.uniform(5, 30, N_ROWS), 0)
    traffic_level = rng.integers(1, 4, N_ROWS)
    rain = rng.binomial(1, 0.25, N_ROWS)
    delivery_min = np.round(
        6.0
        + 3.1 * distance_km
        + 0.65 * prep_time_min
        + 4.2 * traffic_level
        + 5.5 * rain
        + rng.normal(0, 2.5, N_ROWS),
        1,
    )
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(["distance_km", "prep_time_min", "traffic_level",
                    "rain", "delivery_min"])
        for i in range(N_ROWS):
            w.writerow([distance_km[i], int(prep_time_min[i]),
                        int(traffic_level[i]), int(rain[i]), delivery_min
    return path


if not DATA.exists():
    make_delivery_csv()
    print("dataset rebuilt ->", DATA)
```

```
else:
    print("dataset found   ->", DATA)
```

Walkthrough

Read each step, then run its cell.

Step 1 --- Why print() is not enough

Look at what `print` gives you, and what it does not.

In [ ]:
```
print("something went wrong")
```

No time. No severity --- is this fatal or routine? No idea which request, which customer, which co
And no way to turn it off in production without editing the code.

Every one of those is fixed by the logging module.

Step 2 --- Set up a logger

`logging.getLogger(name)` gives you a named logger. We send its output to the screen wi
`StreamHandler`, and decide the format once.

The `handlers.clear()` line matters in notebooks: without it, every time you re-run this cell y
another handler and every message appears twice, then three times.

In [ ]:
```
import logging
import sys
from pathlib import Path

WORK = Path("work")
WORK.mkdir(exist_ok=True)

logger = logging.getLogger("delivery")
logger.setLevel(logging.DEBUG)
logger.handlers.clear()          # re-running this cell is now safe
logger.propagate = False

to_screen = logging.StreamHandler(sys.stdout)
to_screen.setFormatter(logging.Formatter(
    "%(asctime)s  %(levelname)-8s  %(name)s  %(message)s"
))
logger.addHandler(to_screen)

logger.info("logger ready")
```

Step 3 --- The five levels

Run this and read the output. Every line now carries a time and a severity.

In [ ]:
```
logger.debug("feature vector built: [5.0, 20, 2, 0]")
logger.info("prediction served in 3 ms")
logger.warning("distance 47 km is unusually far")
logger.error("model file could not be read")
logger.critical("out of memory, shutting down")
```

Choosing the level is a judgement you will make hundreds of times. The test: **would I want to b up for this?** If yes, ERROR or CRITICAL. If it is just useful history, INFO.

Step 4 --- Turn the volume down without touching the code

This is the payoff of levels. Change one number and the DEBUG and INFO lines disappear --- everywhere, at once.

In [ ]:
```
logger.setLevel(logging.WARNING)

print("--- threshold is now WARNING ---")
logger.debug("you will not see this")
logger.info("nor this")
logger.warning("but you will see this")
logger.error("and this")

logger.setLevel(logging.DEBUG)     # back to chatty for the lab
```

In production you run at INFO. When something is wrong you drop to DEBUG for ten minutes. N rebuild, no redeploy.

Step 5 --- One line, one event, as JSON

Now make the lines machine-readable. A **formatter** decides what a log line looks like; ours build JSON object instead of a sentence.

Anything extra the caller passes lands in the same object, which is what makes the logs searcha

In [ ]:
```
import json
from datetime import datetime, timezone


class JsonFormatter(logging.Formatter):
    """Turn one log record into one line of JSON."""

    def format(self, record):
        event = {
            "time": datetime.fromtimestamp(
                record.created, timezone.utc
            ).isoformat(),
            "level": record.levelname,
            "logger": record.name,
            "message": record.getMessage(),
        }
        # anything passed as extra={...} is added here
        for key, value in getattr(record, "fields", {}).items():
            event[key] = value
        return json.dumps(event)
```

```
def log_event(level, message, **fields):
    """Log one message with named fields attached."""
    logger.log(level, message, extra={"fields": fields})


to_screen.setFormatter(JsonFormatter())

log_event(logging.INFO, "prediction served",
          duration_ms=3, distance_km=5.0, predicted_min=42.1)
```

Read that line. Every fact has a name. A search tool can now answer "average duration_ms yest without anyone writing a parser.

Step 6 --- Send the same events to a file as well

On your laptop it is convenient to keep a copy on disk so you can search it in Step 8. **In a contai would not do this** --- you would log only to the screen and let the platform collect it.

The file format is **JSONL**: one JSON object per line.

In [ ]:
```
LOG_FILE = WORK / "service.log"
LOG_FILE.unlink(missing_ok=True)

to_file = logging.FileHandler(LOG_FILE, encoding="utf-8")
to_file.setFormatter(JsonFormatter())
logger.addHandler(to_file)

log_event(logging.INFO, "service started", version="1.0.0")
print("logging to", LOG_FILE)
```

Step 7 --- Log a service doing its job

Now simulate 40 predictions and log each one properly: what came in, what went out, how long Two of them go wrong on purpose.

Notice `request_id`. When something is reported hours later, that id is how you find the one li among thousands that belongs to it.

In [ ]:
```
import random
import time
import uuid

import joblib
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

FEATURES = ["distance_km", "prep_time_min", "traffic_level", "rain"]
orders = pd.read_csv(DATA)
X_train, X_test, y_train, y_test = train_test_split(
    orders[FEATURES], orders["delivery_min"],
    test_size=0.2, random_state=42,
)
```

```
model = LinearRegression().fit(X_train, y_train)

rng = random.Random(42)
to_screen.setLevel(logging.WARNING)   # keep the notebook readable

for i in range(40):
    request_id = str(uuid.uuid4())[:8]
    row = X_test.iloc[[i]]
    started = time.perf_counter()

    if i in (11, 27):                 # two deliberate failures
        log_event(logging.ERROR, "prediction failed",
                  request_id=request_id,
                  error="ValueError: traffic_level out of range",
                  duration_ms=round(
                      (time.perf_counter() - started) * 1000, 2))
        continue

    minutes = float(model.predict(row)[0])
    duration = (time.perf_counter() - started) * 1000
    if rng.random() < 0.15:           # a few slow ones
        duration += rng.uniform(200, 400)

    log_event(logging.INFO, "prediction served",
              request_id=request_id,
              distance_km=float(row["distance_km"].iloc[0]),
              traffic_level=int(row["traffic_level"].iloc[0]),
              predicted_min=round(minutes, 1),
              duration_ms=round(duration, 2))

to_screen.setLevel(logging.NOTSET)
print("40 requests logged to", LOG_FILE)
print("lines written:",
      len(LOG_FILE.read_text(encoding='utf-8').splitlines()))
```

Only the two errors appeared on screen, because we raised the screen handler's threshold to WARNING. Everything still went to the file. Two destinations, two levels, one line of code each.

Step 8 --- Search the logs like a machine would

This is the whole reason for JSON. Read the file back and answer real questions with ordinary P

In [ ]:
```
events = [json.loads(line)
          for line in LOG_FILE.read_text(encoding="utf-8").splitlines()
          if line.strip()]

print("total events :", len(events))

errors = [e for e in events if e["level"] == "ERROR"]
print("errors       :", len(errors))

served = [e for e in events if e.get("message") == "prediction served"]
durations = [e["duration_ms"] for e in served]
print("predictions  :", len(served))
print("average ms   :", round(sum(durations) / len(durations), 1))

slow = [e for e in served if e["duration_ms"] > 200]
print("slower than 200 ms:", len(slow))
```

```
print()
print("The slowest request:")
worst = max(served, key=lambda e: e["duration_ms"])
print(f"  request_id {worst['request_id']}  "
      f"{worst['duration_ms']} ms  "
      f"{worst['distance_km']} km")
```

Every one of those questions would have been an afternoon of squinting at sentences. That is th difference structure makes.

Step 9 --- What must never go in a log

Logs are kept for months and read by many people. Some things must never be written into one

- names, phone numbers, exact home addresses
- card numbers, passwords, API keys, tokens
- anything identifying one individual customer

You almost never need the value. You need to be able to *match* it. A one-way hash lets you say ' two orders came from the same customer" without recording who that is.

In [ ]:
```
import hashlib

bad_order = {
    "customer_name": "Ananya Sharma",
    "phone": "9876543210",
    "address": "Flat 402, Green Park, Noida",
    "distance_km": 5.0,
    "traffic_level": 2,
}


def safe_for_logging(order):
    """Keep what helps debugging, remove what identifies a person."""
    secret = str(order.get("phone", ""))
    return {
        "customer_ref": hashlib.sha256(
            secret.encode()).hexdigest()[:12],
        "distance_km": order["distance_km"],
        "traffic_level": order["traffic_level"],
    }


print("NEVER log this:")
print("  ", bad_order)
print()
print("Log this instead:")
print("  ", safe_for_logging(bad_order))

log_event(logging.INFO, "order received",
          **safe_for_logging(bad_order))
```

`customer_ref` is stable: the same phone always gives the same reference, so you can still g customer's orders. But nobody reading the log learns a phone number.

Step 10 --- Where logs go once you are in a cluster

Finally, the operational picture from L18. You do not have to run any of this today --- you have to to draw it.

In [ ]:
```
pipeline = [
    ("your code", "writes one JSON line per event to stdout"),
    ("the container runtime", "captures stdout to a file on the node"),
    ("a collector (Fluent Bit / FluentD)",
     "runs on every node and reads those files"),
    ("a store (Elasticsearch / OpenSearch)",
     "indexes the events so they can be searched"),
    ("a viewer (Kibana / Grafana)", "is where a human asks questions"),
]

for i, (part, does) in enumerate(pipeline, 1):
    print(f"  {i}. {part}")
    print(f"     {does}")

print()
print("Two rules that follow from this picture:")
print("  - never write logs to a file inside a container;")
print("    the container dies and takes the file with it")
print("  - never log secrets; that store is searchable by")
print("    everyone in the company")
```

Your turn

The walkthrough above is finished. Now you write some code.

There are **3 tasks**. Each one is small. Each one has a hint. Do them in order.

Where you see `# TODO`, replace that line with your own code. Do not delete the variable name left of the `=` sign --- the self-check at the bottom looks for exactly that name.

When you have tried all three, run the **self-check** cell at the end. It prints a table telling you whic are correct. You can run it as many times as you like.

Task T1 --- Decide what gets recorded

```
A colleague wants the service to record warnings and errors only --
nothing routine.

Set the `logger` threshold so that `DEBUG` and `INFO` are discarded
but
`WARNING`, `ERROR` and `CRITICAL` are kept. Then build a list
`T1_kept` naming the levels that would still be recorded, as upperc
strings.
```

> **Hint.** `logger.setLevel(logging.WARNING)` sets the threshold. For the list, use `logger.isEnabledFor(logging.DEBUG)` and so on to ask the logger itself, rather than typing the answer from memory.

In [ ]:
```
LEVELS = {"DEBUG": logging.DEBUG, "INFO": logging.INFO,
          "WARNING": logging.WARNING, "ERROR": logging.ERROR,
          "CRITICAL": logging.CRITICAL}

# TODO: set the logger threshold so only WARNING and above survive

# TODO: which level names would still be recorded?
T1_kept = None

print("still recorded:", T1_kept)
```

Task T2 --- A formatter for the delivery service

Write a class `ServiceFormatter` (a `logging.Formatter`) whose `format`
method returns one line of JSON with **exactly** these five keys:

- `"ts"` --- the time, as an ISO string
- `"level"` --- the level name
- `"service"` --- always the string `"delivery-api"`
- `"msg"` --- the message
- `"env"` --- always the string `"lab"`

Then create one instance of it, call `format` on a test record, and store the **parsed dictionary** in `T2_line`.

> **Hint.** Copy `JsonFormatter` from Step 5 and change the keys. To make a record to te with, the stub gives you `sample_record`. Call `ServiceFormatter().format(sample_record)` to get a string, then `json.loads(...)` it.

In [ ]:
```
sample_record = logger.makeRecord(
    "delivery", logging.INFO, "x", 1, "prediction served", None, None
)


class ServiceFormatter(logging.Formatter):
    def format(self, record):
        # TODO: return one line of JSON with the five required keys
        return "{}"


# TODO: format the sample record and parse it back into a dictionary
T2_line = None

print(T2_line)
```

Task T3 --- Answer a question from the logs

```
Your manager asks: **"of the predictions we served, what fraction t
longer than 100 milliseconds, and what was the longest one?"**

Using the `events` list from Step 8, work it out:

- `T3_slow_fraction` --- the fraction (between 0 and 1) of
  `"prediction served"` events with `duration_ms` above 100
- `T3_worst_id` --- the `request_id` of the slowest one
```

> **Hint.** Filter `events` down to those whose `message` is `"prediction served"`. Th fraction is `len(slow) / len(served)`. For the slowest, use `max(served, key=...)` on `duration_ms` and read its `request_id`.

In [ ]:
```
served_events = [e for e in events
                 if e.get("message") == "prediction served"]

# TODO: what fraction took longer than 100 ms?
T3_slow_fraction = None

# TODO: the request_id of the slowest one
T3_worst_id = None

print("fraction over 100 ms:", T3_slow_fraction)
print("slowest request     :", T3_worst_id)
```

Self-check

Run the cell below to mark your work.

In [ ]:
```
# ----------------------------------------------------------------
# SELF-CHECK --- run this when you have attempted the tasks above.
# It never breaks your notebook. A task you have not done yet simply
# shows FAIL.
# ----------------------------------------------------------------

_results = []


def _check(label, fn):
    """Evaluate one graded condition without ever raising."""
    try:
        ok = bool(fn())
    except Exception:
        ok = False
    _results.append((label, ok))


_check('T1 | the logger threshold is WARNING', lambda: logger.level == lo
_check('T1 | T1_kept names exactly the three surviving levels', lambda: s
_check('T2 | the line has exactly the five required keys', lambda: set(T2
```

```
_check('T2 | service and env carry the fixed values', lambda: T2_line['se
_check('T2 | level and message come from the record', lambda: T2_line['le
_check('T2 | it still works for a different record', lambda: (lambda d: d
_check('T3 | T3_slow_fraction matches the log file', lambda: abs(float(T3
_check('T3 | T3_worst_id is the slowest served request', lambda: T3_worst

print("======================================================================
print("SELF-CHECK   Practical 10 --- Logs You Can Actually Search")
print("======================================================================
for _label, _ok in _results:
    print(f"  [{'PASS' if _ok else 'FAIL'}]  {_label}")
print("----------------------------------------------------------------------
_passed = sum(1 for _, _ok in _results if _ok)
print(f"  {_passed} of {len(_results)} checks passed")
print("======================================================================
if _passed == len(_results):
    print("Well done. Save the notebook and submit it.")
else:
    print("Read the FAIL lines above, fix those tasks, run this cell agai
```

What to submit

1. This notebook, with every cell run and its output visible.
2. Your `work/service.log`, showing at least 40 JSON lines.
3. In a markdown cell, three fields you would add to every log line in a real delivery service, an each one helps.

Name your file `PXX_<your-roll-number>.ipynb` before you upload it.

How this practical is marked

What is marked	Marks
Walkthrough run end to end, log file produced	3
Task T1 --- levels used to control what is recorded	2
Task T2 --- a formatter with the required fields	3
Task T3 --- a real question answered from the logs	2
Total	**10**

Read more

- Python --- Logging HOWTO --- https://docs.python.org/3/howto/logging.html
- Python --- Logging cookbook --- https://docs.python.org/3/howto/logging-cookbook.html
- Kubernetes --- Logging architecture --- https://kubernetes.io/docs/concepts/cluster-administ logging/
- The twelve-factor app --- Logs --- https://12factor.net/logs

Practical 11 --- A Robot That Checks Your Wor

Continuous integration with GitHub Actions, rehearsed on your own machine

SCSE3040 Machine Learning Operations · Bennett University · Session 2026-27

Lectures this follows	L20-L21
Course Outcome	CO4
Lab duration	120 minutes
Memory needed	about 350 MB (fine on a 4 GB or 8 GB machine)
Extra software	nothing beyond the course venv
Marks	10

Aim

1. Explain what continuous integration is and which problem it solves.
2. Write a GitHub Actions workflow that installs, tests and checks a project.
3. Run the pipeline's own steps on your machine before pushing anything.
4. Read a failed pipeline and say exactly which step broke and why.

What you need to know first

Picture the Friday-evening deploy. Someone pulls the latest code, runs the tests on their laptop, green, and ships. On Monday it is broken, because their laptop had a library the server does not because two of the tests were never actually run.

Continuous integration, usually written **CI**, removes the human from that sentence. Every time pushes code, a clean machine somewhere fetches it, installs everything from scratch, runs every and reports pass or fail. No one has to remember. No one can skip it.

The vocabulary is small.

A **workflow** is the whole automated procedure, written as a YAML file that lives in your repository `.github/workflows/`.

A **trigger** says when to run it --- typically on every push, and on every pull request.

A **job** runs on a fresh machine. A workflow can have several, and they can run at the same time.

A **step** is one command inside a job. Steps run in order, and the first one to fail stops the job.

That last rule is the whole mechanism. Remember from P04 and P05 that a program returns an **code**: 0 for success, anything else for failure. `pytest` returns 1 if any test failed. The pipeline

looks at that number. Your tests from P05 are what makes CI worth having --- without them the r
nothing to check.

Today you will write a real workflow file, and then run its steps on your own machine with a small
we build together. Rehearsing locally is a genuinely professional habit: pushing a broken workflo
waiting three minutes to find out is a slow way to work.

Before you start

- Practical P05 is finished. You can write tests and run pytest.
- You have seen YAML twice now. Spaces only.
- A GitHub account is useful for the optional last step, but nothing today requires one.

How to run a cell

Click on a grey code cell, then press **Shift + Enter**. The cell runs and the cursor moves to the ne
number appears in the `[ ]` on the left when the cell has finished.

Run the cells in order, from the top. A later cell almost always uses something an earlier cell c
you jump ahead you will see a `NameError`, which just means "you have not made that thing y

If everything goes wrong, use the menu: **Kernel -> Restart Kernel and Clear All Outputs**, then
again from the first cell. Nothing is damaged by doing this.

In [ ]:
```
# Step 0 --- check the workbench before we start.
# This cell only looks; it changes nothing. Run it and read the last line

import sys
from pathlib import Path

print("Python  :", sys.version.split()[0])
print("Folder  :", Path.cwd().name)

_missing = []
for _name in ['yaml', 'pytest']:
    try:
        __import__(_name)
    except ImportError:
        _missing.append(_name)

for _name in ['yaml', 'pytest']:
    _mark = "missing" if _name in _missing else "ok"
    print(f"  {_name:<14} {_mark}")

if _missing:
    print()
    print("STOP. Some libraries are missing:", ", ".join(_missing))
    print("Ask your instructor to run the setup in labs/SETUP.md.")
else:
    print()
    print("All good. You can carry on to Step 1.")
```

Step 0b --- the dataset

Every practical in this course uses the same 600 food deliveries. The next cell makes sure the fil
there.

In [ ]:
```python
# The delivery-time dataset every practical in this course uses.
# If the file is missing we build it again from the same seed, so every
# student in the room gets byte-for-byte the same 600 rows.

import csv
from pathlib import Path

import numpy as np

SEED = 42
N_ROWS = 600
DATA = Path("..") / "data" / "delivery_times.csv"


def make_delivery_csv(path=DATA):
    """Write the 600-row delivery dataset. Same formula as the lectures."
    rng = np.random.default_rng(SEED)
    distance_km = np.round(rng.uniform(0.5, 12.0, N_ROWS), 2)
    prep_time_min = np.round(rng.uniform(5, 30, N_ROWS), 0)
    traffic_level = rng.integers(1, 4, N_ROWS)
    rain = rng.binomial(1, 0.25, N_ROWS)
    delivery_min = np.round(
        6.0
        + 3.1 * distance_km
        + 0.65 * prep_time_min
        + 4.2 * traffic_level
        + 5.5 * rain
        + rng.normal(0, 2.5, N_ROWS),
        1,
    )
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(["distance_km", "prep_time_min", "traffic_level",
                    "rain", "delivery_min"])
        for i in range(N_ROWS):
            w.writerow([distance_km[i], int(prep_time_min[i]),
                        int(traffic_level[i]), int(rain[i]), delivery_min
    return path


if not DATA.exists():
    make_delivery_csv()
    print("dataset rebuilt ->", DATA)
else:
    print("dataset found   ->", DATA)
```

Walkthrough

Read each step, then run its cell.

Step 1 --- Build a small project worth checking

CI needs something to test. We reuse the shape from P05: a module, some tests, and a requirer file.

In [ ]:
```
from pathlib import Path

WORK = Path("work")
WORK.mkdir(exist_ok=True)

(WORK / "orders.py").write_text('''"Helpers for delivery orders."

TRAFFIC_LEVELS = (1, 2, 3)


def minutes_per_km(delivery_min, distance_km):
    "How many minutes each kilometre took."
    if distance_km <= 0:
        raise ValueError("distance_km must be positive")
    return delivery_min / distance_km


def is_valid_order(order):
    "True if this order is usable for training."
    return (order.get("distance_km", 0) > 0
            and order.get("prep_time_min", 0) >= 0
            and order.get("traffic_level") in TRAFFIC_LEVELS
            and order.get("rain") in (0, 1))
''', encoding="utf-8")

(WORK / "test_orders.py").write_text('''"Tests for orders.py."

import pytest

from orders import is_valid_order, minutes_per_km


def test_minutes_per_km():
    assert minutes_per_km(30, 10) == 3.0


def test_zero_distance_raises():
    with pytest.raises(ValueError):
        minutes_per_km(30, 0)


def test_good_order():
    assert is_valid_order({"distance_km": 5.0, "prep_time_min": 20,
                           "traffic_level": 2, "rain": 0})
```

```
def test_bad_traffic():
    assert not is_valid_order({"distance_km": 5.0,
                               "prep_time_min": 20,
                               "traffic_level": 9, "rain": 0})
''', encoding="utf-8")

(WORK / "requirements.txt").write_text(
    "pytest==9.1.1\nscikit-learn==1.9.0\npandas==2.3.3\n",
    encoding="utf-8")

for f in sorted(WORK.glob("*")):
    if f.is_file():
        print("  ", f.name)
```

Step 2 --- Write the workflow

This is a real GitHub Actions file. Read it as four questions: **when** does it run, **where** does it run, does it do, and **in what order**.

`uses:` pulls in a ready-made action somebody else wrote --- `checkout` fetches your code, `python` installs Python. `run:` is your own shell command.

The `name:` on each step is what you will see in the web interface, so make them say somethin

In [ ]:
```
workflow = """name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Get the code
        uses: actions/checkout@v4

      - name: Install Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install the libraries
        run: pip install -r requirements.txt

      - name: Run the tests
        run: pytest -q

      - name: Check the config file is valid
        run: python -c "import yaml, pathlib; yaml.safe_load(pathlib.Path
"""

flow_dir = WORK / ".github" / "workflows"
flow_dir.mkdir(parents=True, exist_ok=True)
(flow_dir / "ci.yml").write_text(workflow, encoding="utf-8")
```

```
print(workflow)
```

`runs-on: ubuntu-latest` is worth noticing. The pipeline runs on Linux, on a machine crea seconds ago with nothing installed. That is the point: if it works there, it does not depend on anyt peculiar about your laptop.

Step 3 --- Check the workflow is valid YAML before pushing it

A workflow with a YAML error does not fail loudly --- GitHub often just ignores it, and you wonder nothing ran. Check it yourself.

One oddity to know about: in YAML, a bare `on` is read as the boolean `True`. So after parsing, trigger section appears under the key `True`, not `"on"`. That surprises everybody once.

In [ ]:
```
import yaml

with open(flow_dir / "ci.yml", encoding="utf-8") as fh:
    flow = yaml.safe_load(fh)

print("top-level keys:", list(flow))

triggers = flow.get("on", flow.get(True))
print("triggers      :", list(triggers))
print("jobs          :", list(flow["jobs"]))

steps = flow["jobs"]["test"]["steps"]
print()
print(f"{len(steps)} steps:")
for i, step in enumerate(steps, 1):
    kind = "uses" if "uses" in step else "run"
    print(f"  {i}. [{kind:4}] {step.get('name', '(unnamed)')}")
```

Step 4 --- Build a runner for the steps you can rehearse

`uses:` steps need GitHub's machinery, so we skip those. But every `run:` step is just a shell command --- and those you can rehearse here, in order, stopping at the first failure, exactly as th pipeline would.

One adjustment. On the pipeline's fresh Linux machine, `python` and `pip` mean the right thin your laptop `python` may be a different installation altogether --- on Windows it is often a stub f Microsoft Store. So the runner rewrites those three words to point at the exact interpreter runnin notebook.

That difference **is** the lesson: your machine is not the pipeline's machine, which is why the pipeli exists.

In [ ]:
```
import subprocess
import sys

HERE_PY = f'"{sys.executable}"'
TRANSLATIONS = [
    ("pip", f"{HERE_PY} -m pip"),
```

```
    ("pytest", f"{HERE_PY} -m pytest"),
    ("python", HERE_PY),
]


def for_this_machine(command):
    """Rewrite a CI command so it uses this notebook's Python."""
    for word, replacement in TRANSLATIONS:
        if command == word or command.startswith(word + " "):
            return replacement + command[len(word):]
    return command


def run_pipeline(steps, cwd=WORK):
    """Run the `run:` steps in order and stop at the first failure."""
    results = []
    for step in steps:
        name = step.get("name", "(unnamed)")
        if "run" not in step:
            print(f"[skip] {name}  (needs GitHub: {step['uses']})")
            results.append({"name": name, "status": "skipped"})
            continue

        command = step["run"].strip()
        print(f"[run ] {name}")
        print(f"       $ {command}")
        done = subprocess.run(for_this_machine(command), cwd=cwd,
                              shell=True, capture_output=True,
                              text=True)
        ok = done.returncode == 0
        tail = (done.stdout + done.stderr).strip().splitlines()[-4:]
        for line in tail:
            print("       |", line)
        print(f"       -> exit {done.returncode} "
              f"{'PASS' if ok else 'FAIL'}")
        results.append({"name": name,
                        "status": "pass" if ok else "fail",
                        "exit_code": done.returncode})
        if not ok:
            print()
            print("PIPELINE FAILED. Later steps do not run.")
            break
        print()
    return results


print("runner ready")
```

Step 5 --- Rehearse the pipeline

Run it. The install step uses the libraries you already have, so it is quick; the test step is the one matters.

The last step will fail --- there is no `config.yaml` in this folder. That is deliberate. A pipeline th never failed is a pipeline you have not tested.

In [ ]:
```
results = run_pipeline(steps)
```

```
print("=" * 55)
for r in results:
    print(f"  {r['status']:<8} {r['name']}")
```

Read the failure. It names the step, shows the command, and shows the error. That is exactly wh red cross in GitHub's web interface will show you --- so you can debug it here, in seconds, instea pushing and waiting.

Step 6 --- Fix the pipeline

The pipeline asked for a `config.yaml` that does not exist. Two honest fixes: create the file, o asking for it. We create it, because a real project would have one.

In [ ]:
```
(WORK / "config.yaml").write_text(
    "split:\n  test_size: 0.2\n  seed: 42\n", encoding="utf-8"
)

results = run_pipeline(steps)

print("=" * 55)
passed = sum(1 for r in results if r["status"] == "pass")
failed = sum(1 for r in results if r["status"] == "fail")
print(f"  {passed} passed, {failed} failed")
print("  PIPELINE GREEN" if failed == 0 else "  PIPELINE RED")
```

Step 7 --- Now break a test and watch the robot catch it

This is the whole value of CI in one cell. Introduce a bug the way a tired person would --- a wron operator --- and let the pipeline find it.

In [ ]:
```
original = (WORK / "orders.py").read_text(encoding="utf-8")
(WORK / "orders.py").write_text(
    original.replace("return delivery_min / distance_km",
                     "return delivery_min * distance_km"),
    encoding="utf-8",
)

print("A bug has been introduced. Running the pipeline...")
print()
broken_results = run_pipeline(steps)

(WORK / "orders.py").write_text(original, encoding="utf-8")
print()
print("bug removed, orders.py restored")
```

The pipeline went red at the test step, and the steps after it never ran. On GitHub this is the poin the pull request is blocked and the author gets an email. Nobody had to notice.

Step 8 --- Make the pipeline faster with a cache

Installing libraries from scratch on every push is slow. GitHub can keep pip's download cache be runs; `setup-python` does it with one extra line.

Speed is not vanity here. A pipeline that takes fifteen minutes is a pipeline people learn to ignore

In [ ]:
```
cached = yaml.safe_load(
    (flow_dir / "ci.yml").read_text(encoding="utf-8")
)
for step in cached["jobs"]["test"]["steps"]:
    if step.get("uses", "").startswith("actions/setup-python"):
        step["with"]["cache"] = "pip"

(flow_dir / "ci-cached.yml").write_text(
    yaml.safe_dump(cached, sort_keys=False), encoding="utf-8"
)

print(yaml.safe_dump(cached["jobs"]["test"]["steps"][1],
                     sort_keys=False))
```

Step 9 --- What happens on GitHub

You do not have to do this today, but you should know the shape of it.

In [ ]:
```
print("To make this real:")
print()
print("  1. Put your project in a GitHub repository.")
print("  2. Commit the file at .github/workflows/ci.yml")
print("     -- the path must be exactly that.")
print("  3. git push")
print("  4. Open the repository in a browser and click the")
print("     'Actions' tab at the top.")
print()
print("What you will see first: a list of workflow runs, newest")
print("at the top, each with a yellow dot (running), a green tick")
print("(passed) or a red cross (failed). Click a run, then a job,")
print("to expand each step and read its output.")
print()
print("In Settings -> Branches you can then add a rule that")
print("REQUIRES this check to pass before anyone may merge into")
print("main. That is the moment CI stops being advice and starts")
print("being a rule.")
```

Your turn

The walkthrough above is finished. Now you write some code.

There are **3 tasks**. Each one is small. Each one has a hint. Do them in order.

Where you see `# TODO`, replace that line with your own code. Do not delete the variable name left of the `=` sign --- the self-check at the bottom looks for exactly that name.

When you have tried all three, run the **self-check** cell at the end. It prints a table telling you whic are correct. You can run it as many times as you like.

Task T1 --- Trigger it on the right events

Build a dictionary `T1_triggers` describing a workflow that runs:

- on every **push** to the branches `main` **and** `develop`
- on every **pull request** targeting `main`

The shape must match what GitHub expects, that is: `{"push": {"branches": [...]}, "pull_request": {"branches": [...]}}`

> **Hint.** This is a plain Python dictionary; no YAML needed. Look at the `on:` block printed Step 2 and write the same structure with one extra branch under `push`.

In [ ]:
```
# TODO: push on main and develop, pull_request on main
T1_triggers = None

print(T1_triggers)
```

Task T2 --- Add a step that would have caught a real bug

Add one more step to the workflow, named exactly `Check the model can be imported`, runs the command:

```
python -c "import orders; print(orders.minutes_per_km(30, 10))"
```

Append it to the `steps` list, write the updated workflow back to `work/.github/workflows/ci.yml`, and store the new steps list in `T2_steps`.

> **Hint.** A step is a dictionary with `name` and `run` keys. Use `steps.append({...})`, then rebuild the file with `yaml.safe_dump(flow, sort_keys=False)`. Set `T2_steps = steps`.

In [ ]:
```
new_step = {
    "name": "Check the model can be imported",
    # TODO: the command to run
    "run": None,
}

# TODO: append it to steps, save the workflow file, and set T2_steps
T2_steps = None

print(T2_steps)
```

Task T3 --- Rehearse it, and get it green

Run your six-step pipeline with `run_pipeline` and store the result list in `T3_results`.

Every step that actually runs must pass. If something fails, read the output, fix it, and run again - as you would before pushing.

> **Hint.** `T3_results = run_pipeline(T2_steps)`. If the import check fails with `ModuleNotFoundError`, check that `orders.py` is still intact -- Step 7 modified it a put it back.

In [ ]:
```
# TODO: run the pipeline and keep the results
T3_results = None

print(T3_results)
```

Self-check

Run the cell below to mark your work.

In [ ]:
```
# ---------------------------------------------------------------------
# SELF-CHECK --- run this when you have attempted the tasks above.
# It never breaks your notebook. A task you have not done yet simply
# shows FAIL.
# ---------------------------------------------------------------------

_results = []


def _check(label, fn):
    """Evaluate one graded condition without ever raising."""
    try:
        ok = bool(fn())
    except Exception:
        ok = False
    _results.append((label, ok))


_check('T1 | both trigger kinds are present', lambda: set(T1_triggers) ==
_check('T1 | push covers main and develop', lambda: sorted(T1_triggers['p
_check('T1 | pull_request targets main only', lambda: T1_triggers['pull_r
_check('T2 | the workflow now has six steps', lambda: len(T2_steps) == 6)
_check('T2 | the last one is the import check, with a run command', lambd
_check('T2 | its command really imports orders and calls the function', l
_check('T2 | the change was saved to the workflow file', lambda: 'Check t
_check('T3 | the pipeline ran all six steps', lambda: len(T3_results) ==
_check('T3 | nothing failed', lambda: not any(r['status'] == 'fail' for r
_check('T3 | the tests and the import check both actually ran', lambda: s

print("=====================================================================
print("SELF-CHECK   Practical 11 --- A Robot That Checks Your Work")
print("=====================================================================
for _label, _ok in _results:
    print(f"  [{'PASS' if _ok else 'FAIL'}]  {_label}")
print("---------------------------------------------------------------------
_passed = sum(1 for _, _ok in _results if _ok)
print(f"  {_passed} of {len(_results)} checks passed")
print("=====================================================================
if _passed == len(_results):
    print("Well done. Save the notebook and submit it.")
```

```
else:
    print("Read the FAIL lines above, fix those tasks, run this cell agai
```

What to submit

1. This notebook, with every cell run and its output visible.
2. Your `work/.github/workflows/ci.yml`.
3. In a markdown cell, two sentences: what the pipeline did when you broke the test, and why after it did not run.

Name your file `PXX_<your-roll-number>.ipynb` before you upload it.

How this practical is marked

What is marked	Marks
Walkthrough run end to end, pipeline goes green then red	3
Task T1 --- correct triggers on the workflow	2
Task T2 --- a useful step added to the pipeline	3
Task T3 --- the pipeline rehearsed and green	2
Total	**10**

Read more

- GitHub Docs --- Understanding GitHub Actions --- https://docs.github.com/en/actions/learn-g actions/understanding-github-actions
- GitHub Docs --- Workflow syntax --- https://docs.github.com/en/actions/reference/workflow-s for-github-actions
- GitHub Docs --- Building and testing Python --- https://docs.github.com/en/actions/automatir and-tests/building-and-testing-python
- GitHub Docs --- About protected branches --- https://docs.github.com/en/repositories/configu branches-and-merges-in-your-repository/managing-protected-branches/about-protected-bra